SÁCH

Khoa học máy tính

Nhập môn Tính toán và Lập trình sử dụng Python: Ứng dụng trong việc Thấu hiểu Dữ liệu

TP. Hồ Chí Minh, tháng 03 năm 2026

MỤC LỤC

MỤC LỤC **1**

1. BẮT ĐẦU **2**

2. NHẬP MÔN PYTHON **7**

2.1 Các thành phần cơ bản của Python 8

2.1.1 Đối tượng, Biểu thức và các Kiểu số học 9

2.1.2 Biến và Phép gán 11

2.1.3 Các trình thông dịch (IDE) của Python 13

2.2 Chương trình rẽ nhánh 14

2.3 Chuỗi ký tự và Nhập liệu 17

2.3.1 Nhập liệu (Input) 18

2.3.2 Chú giải về Mã hóa ký tự (Character Encoding) 19

2.4 Vòng lặp 20

3. MỘT SỐ CHƯƠNG TRÌNH SỐ HỌC ĐƠN GIẢN **23**

3.1 Liệt kê vét cạn (Exhaustive Enumeration) 23

3.2 Vòng lặp For 25

3.3 Nghiệm xấp xỉ và Tìm kiếm nhị phân (Bisection Search) 27

3.4 Một vài lưu ý khi sử dụng số thực dấu phẩy động (Floats) 31

3.5 Phương pháp Newton-Raphson 33

1. BẮT ĐẦU

Một chiếc máy tính chỉ làm hai việc, và chỉ hai việc mà thôi: nó thực hiện các phép tính và ghi nhớ kết quả của những phép tính đó. Nhưng nó thực hiện hai việc này cực kỳ tốt. Một chiếc máy tính thông thường đặt trên bàn hoặc trong cặp có thể thực hiện khoảng một tỷ phép tính mỗi giây. Thật khó để hình dung tốc độ đó thực sự nhanh đến mức nào. Hãy thử tưởng tượng bạn cầm một quả bóng cách sàn nhà một mét và thả nó ra. Trước khi quả bóng chạm sàn, máy tính của bạn đã có thể thực hiện hơn một tỷ chỉ lệnh.

Về bộ nhớ, một chiếc máy tính nhỏ có thể có hàng trăm gigabyte lưu trữ. Con số đó lớn như thế nào? Nếu một byte (số bit, thường là tám, cần thiết để biểu diễn một ký tự) nặng một gam (thực tế thì không phải vậy), thì 100 gigabyte sẽ nặng 10.000 tấn hệ mét. Để so sánh, con số đó xấp xỉ tổng trọng lượng của 15.000 con voi châu Phi.

Trong phần lớn lịch sử nhân loại, việc tính toán bị giới hạn bởi tốc độ tính toán của bộ não con người và khả năng ghi chép kết quả bằng tay. Điều này có nghĩa là chỉ những vấn đề nhỏ nhất mới có thể được giải quyết bằng phương pháp tính toán. Ngay cả với tốc độ của máy tính hiện đại, vẫn có những vấn đề nằm ngoài khả năng của các mô hình tính toán hiện nay (ví dụ: hiểu về biến đổi khí hậu), nhưng ngày càng có nhiều vấn đề đang chứng minh được tính khả thi đối với các giải pháp tính toán. Hy vọng rằng khi đọc xong cuốn sách này, bạn sẽ cảm thấy thoải mái khi áp dụng tư duy máy tính để giải quyết nhiều vấn đề gặp phải trong quá trình học tập, làm việc và ngay cả trong cuộc sống hàng ngày.

Tư duy máy tính (Computational thinking) có nghĩa là gì?

Mọi kiến thức đều có thể được coi là dạng khai báo (declarative) hoặc mệnh lệnh (imperative). **Kiến thức khai báo** bao gồm các phát biểu về sự thật. Ví dụ: "Căn bậc hai của x là một số y sao cho $y * y = x$." Đây là một phát biểu sự thật. Tiếc thay, nó không cho chúng ta biết bất cứ điều gì về cách để tìm ra căn bậc hai.

Kiến thức mệnh lệnh là kiến thức về "cách làm", hoặc các "công thức" (recipes) để suy luận thông tin. Heron của Alexandria là người đầu tiên ghi lại phương pháp tính căn bậc hai của một số. Phương pháp tìm căn bậc hai của một số (gọi là x) của ông có thể được tóm tắt như sau:

- Bắt đầu với một giá trị dự đoán, g.
- Nếu $g * g$ đủ gần với x, dừng lại và kết luận g là đáp án.
- Nếu không, tạo một giá trị dự đoán mới bằng cách lấy trung bình cộng của g và x/g, tức là $(g + x/g)/2$.
- Sử dụng giá trị dự đoán mới này (vẫn gọi là g), lặp lại quá trình cho đến khi $g * g$ đủ gần với x.

Xét ví dụ tìm căn bậc hai của 25:

- Đặt g bằng một giá trị bất kỳ, ví dụ: 3.
- Chúng ta xác định rằng $3 * 3 = 9$ là chưa đủ gần với 25.
- Đặt g bằng $(3 + 25/3)/2 = 5,67$.
- Chúng ta xác định rằng $5,67 * 5,67 = 32,15$ vẫn chưa đủ gần với 25.
- Đặt g bằng $(5,67 + 25/5,67)/2 = 5,04$.
- Chúng ta xác định rằng $5,04 * 5,04 = 25,4$ là đủ gần, nên dừng lại và tuyên bố 5,04 là một xấp xỉ phù hợp cho căn bậc hai của 25.

Lưu ý rằng mô tả về phương pháp này là một chuỗi các bước đơn giản, cùng với một luồng kiểm soát xác định thời điểm thực hiện mỗi bước. Một mô tả như vậy được gọi là **thuật toán (algorithm)**. Thuật toán này là một ví dụ về thuật toán "dự đoán và kiểm tra" (guess-and-check). Nó dựa trên thực tế là việc kiểm tra xem một dự đoán có tốt hay không là rất dễ dàng.

Một cách chính thức hơn, thuật toán là một danh sách hữu hạn các chỉ lệnh mô tả một **quá trình tính toán** mà khi thực thi trên một tập hợp dữ liệu đầu vào, nó sẽ đi qua một tập hợp các trạng thái được xác định rõ ràng và cuối cùng tạo ra kết quả đầu ra.

Thuật toán hơi giống một công thức nấu ăn:

- Đặt hỗn hợp kem trứng (custard) lên bếp nóng.
- Khuấy.
- Nhúng thìa vào kem trứng.
- Nhấc thìa ra và dùng ngón tay quẹt qua mặt sau của thìa.
- Nếu để lại một vệt sạch, nhấc kem trứng khỏi bếp và để nguội.
- Nếu không, lặp lại.

Nó bao gồm một số phép thử để quyết định khi nào quá trình hoàn tất, cũng như các chỉ lệnh về thứ tự thực hiện, đôi khi sẽ nhảy đến một chỉ lệnh cụ thể dựa trên kết quả của phép thử.

Vậy làm thế nào để hiện thực hóa ý tưởng về một "công thức" vào một quy trình cơ khí? Một cách là thiết kế một chiếc máy chuyên biệt chỉ dành riêng cho việc tính căn bậc hai. Nghe có vẻ kỳ lạ, nhưng những chiếc máy tính đầu tiên thực chất là **các máy tính chương trình cố định (fixed-program computers)**. Điều này có nghĩa là chúng được thiết kế để thực hiện những công việc rất cụ thể, và chủ yếu là công cụ để giải một bài toán toán học nhất định, ví dụ như tính toán quỹ đạo của một quả đạn pháo. Một trong những chiếc máy tính đầu tiên (được xây dựng năm 1941 bởi Atanasoff và Berry) đã giải các hệ phương trình tuyến tính, nhưng không thể làm gì khác.

Cỗ máy Bombe của Alan Turing, được phát triển trong Thế chiến II, được thiết kế nghiêm ngặt cho mục đích giải mã Enigma của Đức. Một số máy tính rất đơn giản ngày nay vẫn sử dụng cách tiếp cận này. Ví dụ, một chiếc máy tính cầm tay đơn giản là một máy tính chương trình cố định. Nó có thể thực hiện các phép tính số học cơ bản, nhưng không thể dùng làm trình soạn thảo văn bản hay để chơi điện tử. Để thay đổi chương trình của một chiếc máy như vậy, người ta phải thay thế toàn bộ hệ thống mạch điện.

Chiếc máy tính thực sự hiện đại đầu tiên là Manchester Mark 1. Nó khác biệt với các thế hệ tiền bối bởi thực tế nó là một **máy tính lưu trữ chương trình (stored-program computer)**. Loại máy tính này lưu trữ (và xử lý) một chuỗi các chỉ lệnh, và có các thành phần có khả năng thực thi bất kỳ chỉ lệnh nào trong chuỗi đó. Bằng cách tạo ra một kiến trúc tập chỉ lệnh (instruction-set architecture) và chi tiết hóa việc tính toán dưới dạng một chuỗi các chỉ lệnh (tức là một chương trình), chúng ta có được một cỗ máy cực kỳ linh hoạt.

Bằng cách xử lý các chỉ lệnh đó giống như dữ liệu, một máy tính lưu trữ chương trình có thể dễ dàng thay đổi chương trình, và thực hiện việc đó dưới sự kiểm soát của chính chương trình. Thật vậy, "trái tim" của máy tính khi đó trở thành một chương trình (được gọi là **trình thông dịch - interpreter**) có khả năng thực thi bất kỳ tập hợp chỉ lệnh hợp lệ nào, và do đó có thể được sử dụng để tính toán bất cứ thứ gì mà người ta có thể mô tả bằng một tập hợp các chỉ lệnh cơ bản.

Cả chương trình và dữ liệu mà nó xử lý đều nằm trong bộ nhớ. Thông thường, có một bộ đếm chương trình (program counter) trỏ đến một vị trí cụ thể trong bộ nhớ, và quá trình tính toán bắt đầu bằng việc thực thi chỉ lệnh tại điểm đó. Đa số trường hợp, trình thông dịch chỉ đơn giản đi đến chỉ lệnh tiếp theo trong chuỗi, nhưng không phải lúc nào cũng vậy. Trong một số trường hợp, nó thực hiện một phép thử, và dựa trên kết quả của phép thử đó, việc thực thi có thể nhảy đến một điểm khác trong chuỗi chỉ lệnh. Đây được gọi là **luồng điều khiển (flow of control)**, và nó là yếu tố thiết yếu cho phép chúng ta viết các chương trình thực hiện được những tác vụ phức tạp.

Quay lại với phép ẩn dụ về công thức nấu ăn: với một tập hợp nguyên liệu cố định, một đầu bếp giỏi có thể tạo ra vô số món ăn ngon bằng cách kết hợp chúng theo những cách khác nhau. Tương tự như vậy, với một tập hợp nhỏ các tính năng sơ khai cố định, một lập trình viên giỏi có thể tạo ra vô số chương trình hữu ích. Đây chính là điều khiến lập trình trở thành một nỗ lực tuyệt vời.

Để tạo ra các "công thức", hay các chuỗi chỉ lệnh, chúng ta cần một **ngôn ngữ lập trình** để mô tả chúng—một cách để đưa ra "mệnh lệnh hành quân" cho máy tính. Năm 1936, nhà toán học người Anh Alan Turing đã mô tả một thiết bị tính toán giả định mà sau này được gọi là **Máy Turing vạn năng (Universal Turing Machine)**. Cỗ máy này có một bộ nhớ không giới hạn dưới dạng một "dải băng" mà người ta có thể ghi các số 0 và 1, cùng với một số chỉ lệnh sơ khai đơn giản để di chuyển, đọc và ghi lên dải băng đó. Luận đề Church-Turing khẳng định rằng nếu một hàm số là có thể tính toán được (computable), thì một Máy Turing có thể được lập trình để tính toán nó.

Chữ "nếu" trong luận đề Church-Turing rất quan trọng. Không phải mọi bài toán đều có giải pháp tính toán. Ví dụ, Turing đã chứng minh rằng không thể viết một chương trình mà khi đưa vào một chương trình *P* bất kỳ, nó sẽ in ra "true" khi và chỉ khi *P* sẽ chạy mãi mãi. Đây được gọi là **bài toán dừng (halting problem)**.

Luận đề Church-Turing dẫn trực tiếp đến khái niệm **tính đầy đủ Turing (Turing completeness)**. Một ngôn ngữ lập trình được coi là đầy đủ Turing nếu nó có thể được dùng để mô phỏng một Máy Turing vạn năng. Tất cả các ngôn ngữ lập trình hiện đại đều đầy đủ Turing. Hệ quả là:

- bất cứ thứ gì có thể lập trình được bằng ngôn ngữ này (ví dụ: Python) đều có thể lập trình được bằng bất kỳ ngôn ngữ nào khác (ví dụ: Java).

Tất nhiên, một số thứ có thể dễ lập trình hơn trong một ngôn ngữ cụ thể, nhưng về cơ bản, tất cả các ngôn ngữ đều bình đẳng về sức mạnh tính toán.

May mắn thay, không lập trình viên nào phải xây dựng chương trình từ các chỉ lệnh sơ khai của Turing. Thay vào đó, các ngôn ngữ lập trình hiện đại cung cấp một tập hợp các lệnh sơ khai lớn hơn và

tiện lợi hơn. Tuy nhiên, ý tưởng cốt lõi của lập trình—xem đó là quá trình lắp ráp một chuỗi các thao tác—vẫn luôn là trọng tâm.

Dù bạn có tập hợp các lệnh sơ khai nào, hay bất kể phương pháp sử dụng chúng ra sao, thì điều tốt nhất và điều tồi tệ nhất về lập trình đều giống nhau: máy tính sẽ thực hiện chính xác những gì bạn bảo nó làm. Đây là một điều tốt vì nó có nghĩa là bạn có thể khiến máy tính thực hiện mọi thứ thú vị và hữu ích. Nhưng đó cũng là một điều tồi tệ vì khi nó không làm đúng ý bạn, bạn thường chẳng thể đổ lỗi cho ai khác ngoài chính mình.

Có hàng trăm ngôn ngữ lập trình trên thế giới. Không có ngôn ngữ nào là "tốt nhất" (mặc dù người ta có thể đề cử một vài ứng viên cho danh hiệu "tệ nhất"). Các ngôn ngữ khác nhau sẽ phù hợp hơn hoặc kém hơn cho các loại ứng dụng khác nhau. Ví dụ:

- MATLAB là một ngôn ngữ tốt để xử lý các vectơ và ma trận.
- C là một ngôn ngữ tốt để viết các chương trình điều khiển mạng dữ liệu.
- PHP là một ngôn ngữ tốt để xây dựng các trang web.
- Và Python là một ngôn ngữ vạn năng (general-purpose) tốt.

Mỗi ngôn ngữ lập trình đều có một tập hợp các cấu trúc sơ khai (primitive constructs), cú pháp (syntax), ngữ nghĩa tĩnh (static semantics) và ngữ nghĩa (semantics). Tương tự như một ngôn ngữ tự nhiên (ví dụ: tiếng Anh), các cấu trúc sơ khai là các từ, cú pháp mô tả chuỗi từ nào tạo thành các câu đúng cấu trúc, ngữ nghĩa tĩnh xác định câu nào có ý nghĩa, và ngữ nghĩa xác định ý nghĩa của những câu đó. Các cấu trúc sơ khai trong Python bao gồm **các giá trị trực tiếp** (**literals** - ví dụ: số 3.2 và chuỗi 'abc') và **các toán tử trung tố** (**infix operators** - ví dụ: + và /).

Cú pháp của một ngôn ngữ xác định chuỗi ký tự và biểu tượng nào là đúng cấu trúc. Ví dụ, trong tiếng Anh, chuỗi "Cat dog boy." không phải là một câu hợp lệ về mặt cú pháp, vì cú pháp tiếng Anh không chấp nhận các câu có dạng <Danh từ> <Danh từ> <Danh từ>. Trong Python, trình tự các lệnh sơ khai 3.2 + 3.2 là đúng cấu trúc cú pháp, nhưng trình tự 3.2 3.2 thì không.

Ngữ nghĩa tĩnh xác định những chuỗi đúng cú pháp nào thực sự mang một ý nghĩa. Ví dụ trong tiếng Anh, chuỗi "I runs fast" có dạng <Đại từ> <Động từ> <Trạng từ>, vốn là một trình tự được chấp nhận về mặt cú pháp. Tuy nhiên, nó không phải là tiếng Anh chuẩn vì danh từ "I" (tôi) là số ít nhưng động từ "runs" lại chia ở dạng số nhiều. Đây là một ví dụ về lỗi ngữ nghĩa tĩnh. Trong Python, trình tự 3.2 / 'abc' là đúng cú pháp (dạng <Số> / <Chuỗi>), nhưng lại tạo ra lỗi ngữ nghĩa tĩnh vì việc chia một con số cho một chuỗi ký tự là không có ý nghĩa.

Ngữ nghĩa của một ngôn ngữ gắn kết một ý nghĩa cụ thể với mỗi chuỗi biểu tượng đúng cú pháp và không có lỗi ngữ nghĩa tĩnh. Trong ngôn ngữ tự nhiên, ngữ nghĩa của một câu có thể mơ hồ. Ví dụ, câu "Tôi không thể khen ngợi sinh viên này quá lời" (I cannot praise this student too highly) có thể là một lời khen ngợi hết lời hoặc một lời chỉ trích nặng nề. Ngôn ngữ lập trình được thiết kế sao cho mỗi chương trình hợp lệ chỉ có duy nhất một ý nghĩa.

Mặc dù lỗi cú pháp là loại lỗi phổ biến nhất (đặc biệt là đối với những người mới học một ngôn ngữ lập trình mới), chúng lại là loại lỗi ít nguy hiểm nhất. Mọi ngôn ngữ lập trình nghiêm túc đều thực hiện triệt để việc phát hiện các lỗi cú pháp và sẽ không cho phép người dùng thực thi chương trình nếu có dù chỉ một lỗi cú pháp. Hơn nữa, trong hầu hết các trường hợp, hệ thống ngôn ngữ đưa ra chỉ dẫn đủ rõ ràng về vị trí của lỗi để việc sửa lỗi trở nên hiển nhiên.

Tình hình đối với các lỗi ngữ nghĩa tĩnh có phần phức tạp hơn. Một số ngôn ngữ lập trình, ví dụ như Java, thực hiện rất nhiều hoạt động kiểm tra ngữ nghĩa tĩnh trước khi cho phép một chương trình được thực thi. Những ngôn ngữ khác, ví dụ như C và Python (thật đáng tiếc), thực hiện tương đối ít việc kiểm tra ngữ nghĩa tĩnh trước khi chương trình chạy. Tuy nhiên, Python thực hiện một lượng đáng kể việc kiểm tra ngữ nghĩa trong khi đang chạy chương trình.

Người ta thường không nói rằng một chương trình có lỗi ngữ nghĩa. Nếu một chương trình không có lỗi cú pháp và không có lỗi ngữ nghĩa tĩnh, nó mang một ý nghĩa, tức là nó có ngữ nghĩa (semantics). Tất nhiên, điều đó không có nghĩa là nó mang ngữ nghĩa mà người tạo ra nó mong muốn. Khi một chương trình có ý nghĩa khác với những gì người tạo ra nó nghĩ, những điều tồi tệ có thể xảy ra.

Điều gì có thể xảy ra nếu chương trình có lỗi và hành xử theo cách không mong muốn?

- Nó có thể bị sập (crash): tức là ngừng chạy và đưa ra một dấu hiệu hiển nhiên nào đó cho thấy nó đã bị dừng. Trong một hệ thống máy tính được thiết kế đúng cách, khi một chương trình bị sập, nó không gây hư hại cho toàn bộ hệ thống. Tất nhiên, một số hệ thống máy tính rất phổ biến không có đặc tính tốt đẹp này. Hầu như bất kỳ ai sử dụng máy tính cá nhân đều từng chạy một chương trình khiến họ buộc phải khởi động lại toàn bộ máy tính.
- Hoặc nó có thể tiếp tục chạy mãi mãi: chạy và chạy mà không bao giờ dừng lại. Nếu bạn không biết xấp xỉ thời gian chương trình cần để hoàn thành công việc, tình huống này có thể rất khó nhận ra.
- Hoặc nó có thể chạy cho đến khi hoàn thành: nhưng lại tạo ra một kết quả có thể đúng, hoặc không.

Mỗi trường hợp trên đều tệ, nhưng trường hợp cuối cùng chắc chắn là tồi tệ nhất. Khi một chương trình có vẻ như đang làm đúng nhưng thực tế thì không, những hậu quả thảm khốc có thể theo sau: tài sản có thể bị thất thoát, bệnh nhân có thể nhận liều xạ trị gây tử vong, máy bay có thể rơi.

Bất cứ khi nào có thể, các chương trình nên được viết theo cách mà khi chúng hoạt động không đúng, lỗi đó phải tự hiển hiện rõ ràng. Chúng ta sẽ thảo luận về cách thực hiện điều này xuyên suốt cuốn sách.

Bài tập nhỏ: Máy tính có thể máy móc đến mức gây khó chịu. Nếu bạn không nói cho chúng chính xác những gì bạn muốn chúng làm, chúng có khả năng cao sẽ làm sai. Hãy thử viết một thuật toán để lái xe giữa hai địa điểm. Hãy viết nó theo cách bạn hướng dẫn cho một người bình thường, sau đó tưởng tượng điều gì sẽ xảy ra nếu người đó "ngốc" như một chiếc máy tính và thực thi thuật toán chính xác như những gì được viết. Người đó có thể sẽ phải nhận bao nhiêu phiếu phạt giao thông?

2. NHẬP MÔN PYTHON

Mặc dù mỗi ngôn ngữ lập trình đều khác nhau (tuy không khác biệt nhiều như các nhà thiết kế ngôn ngữ muốn chúng ta tin tưởng), vẫn có một số tiêu chí để liên kết chúng lại với nhau:

- **Bậc thấp so với bậc cao (Low-level versus high-level):** Đề cập đến việc chúng ta lập trình bằng các chỉ lệnh và đối tượng dữ liệu ở cấp độ máy (ví dụ: di chuyển 64 bit dữ liệu từ vị trí này sang vị trí khác) hay lập trình bằng các thao tác trừu tượng hơn (ví dụ: làm xuất hiện một menu trên màn hình) do nhà thiết kế ngôn ngữ cung cấp.
- **Đa năng so với Chuyên biệt (General versus targeted to an application domain):** Đề cập đến việc các thao tác sơ khai của ngôn ngữ lập trình có khả năng ứng dụng rộng rãi hay được tinh chỉnh cho một miền cụ thể. Ví dụ, SQL được thiết kế để tạo thuận lợi cho việc trích xuất thông tin từ các cơ sở dữ liệu quan hệ, nhưng bạn sẽ không muốn sử dụng nó để xây dựng một hệ điều hành.
- **Thông dịch so với Biên dịch (Interpreted versus compiled):** Đề cập đến việc chuỗi các chỉ lệnh do lập trình viên viết, gọi là mã nguồn (source code), được thực thi trực tiếp (bởi một trình thông dịch) hay trước tiên nó được chuyển đổi (bởi một trình biên dịch) thành một chuỗi các thao tác sơ khai cấp độ máy. (Vào thời kỳ đầu của máy tính, con người phải viết mã nguồn bằng một ngôn ngữ rất gần với mã máy để có thể được phần cứng máy tính thông dịch trực tiếp). Cả hai phương pháp đều có ưu điểm. Thông thường, việc gỡ lỗi (debug) các chương trình viết bằng ngôn ngữ thông dịch sẽ dễ dàng hơn, vì trình thông dịch có thể tạo ra các thông báo lỗi dễ dàng đối chiếu với mã nguồn. Các ngôn ngữ biên dịch thường tạo ra các chương trình chạy nhanh hơn và tốn ít không gian lưu trữ hơn.

Trong cuốn sách này, chúng ta sử dụng Python. Tuy nhiên, cuốn sách này không phải nói về Python. Nó chắc chắn sẽ giúp người đọc học Python, và đó là một điều tốt. Tuy nhiên, điều quan trọng hơn nhiều là những người đọc cẩn thận sẽ học được cách viết chương trình để giải quyết các vấn đề. Kỹ năng này có thể được chuyển đổi sang bất kỳ ngôn ngữ lập trình nào khác.

Python là một ngôn ngữ lập trình đa năng, có thể được sử dụng hiệu quả để xây dựng hầu hết mọi loại chương trình mà không cần truy cập trực tiếp vào phần cứng máy tính. Python không phải là lựa chọn tối ưu cho các chương trình có yêu cầu cao về độ tin cậy (do cơ chế kiểm tra ngữ nghĩa tĩnh yếu) hoặc các chương trình được xây dựng và bảo trì bởi rất nhiều người hoặc trong một thời gian dài (cũng vì lý do kiểm tra ngữ nghĩa tĩnh yếu).

Tuy nhiên, Python có một số lợi thế so với nhiều ngôn ngữ khác:

- Nó là một ngôn ngữ tương đối đơn giản và dễ học.
- Vì Python được thiết kế để thông dịch, nó có thể cung cấp các phản hồi trong thời gian chạy (runtime feedback) đặc biệt hữu ích cho những lập trình viên mới bắt đầu.

- Ngoài ra còn có một số lượng lớn các thư viện miễn phí kết nối với Python và cung cấp các chức năng mở rộng hữu ích. Một vài trong số đó sẽ được sử dụng trong cuốn sách này.

Bây giờ chúng ta đã sẵn sàng để bắt đầu học một số yếu tố cơ bản của Python. Về mặt khái niệm, những yếu tố này là chung cho hầu hết tất cả các ngôn ngữ lập trình, mặc dù không nhất thiết phải giống nhau về chi tiết.

Người đọc cần được cảnh báo trước rằng cuốn sách này hoàn toàn không phải là một tài liệu giới thiệu toàn diện về Python. Chúng tôi sử dụng Python như một phương tiện để trình bày các khái niệm liên quan đến việc giải quyết vấn đề và tư duy máy tính. Ngôn ngữ này được trình bày dần dần theo từng phần nhỏ khi cần thiết cho mục đích chính nêu trên. Những tính năng của Python mà chúng tôi không cần cho mục đích đó sẽ không được trình bày. Chúng tôi cảm thấy thoải mái về việc không bao quát toàn bộ ngôn ngữ vì đã có những nguồn tài liệu trực tuyến xuất sắc mô tả hầu hết mọi khía cạnh của ngôn ngữ này. Khi chúng tôi giảng dạy khóa học dựa trên cuốn sách này, chúng tôi khuyên sinh viên nên dựa vào các nguồn tài tuyến miễn phí này để làm tài liệu tham khảo về Python.

Python là một ngôn ngữ sống. Kể từ khi được giới thiệu bởi Guido van Rossum vào năm 1990, nó đã trải qua nhiều thay đổi. Trong thập kỷ đầu tiên, Python là một ngôn ngữ ít được biết đến và ít được sử dụng. Điều đó đã thay đổi với sự ra đời của Python 2.0 vào năm 2000. Ngoài việc tích hợp một số cải tiến quan trọng cho chính ngôn ngữ, nó đã đánh dấu một bước chuyển mình trong lộ trình tiến hóa của ngôn ngữ này. Một lượng lớn người dùng bắt đầu phát triển các thư viện kết nối liền mạch với Python, và việc hỗ trợ cũng như phát triển hệ sinh thái Python liên tục đã trở thành một hoạt động dựa trên cộng đồng. Python 3.0 được phát hành vào cuối năm 2008. Phiên bản này đã dọn dẹp nhiều điểm thiếu nhất quán trong thiết kế của các bản phát hành Python 2 khác nhau (thường được gọi là Python 2.x). Tuy nhiên, nó không có tính tương thích ngược (backward compatible). Điều đó có nghĩa là hầu hết các chương trình được viết cho các phiên bản Python trước đó không thể chạy bằng các trình thực thi của Python 3.

Trong vài năm qua, hầu hết các thư viện Python công cộng quan trọng đã được chuyển sang Python 3 và được kiểm thử kỹ lưỡng bằng Python 3.5—phiên bản Python mà chúng tôi sử dụng trong cuốn sách này.

2.1 Các thành phần cơ bản của Python

Một chương trình Python, đôi khi được gọi là một tập lệnh (script), là một chuỗi các định nghĩa và lệnh. Các định nghĩa này được tính toán và các lệnh được thực thi bởi trình thông dịch Python bên trong một môi trường gọi là shell. Thông thường, một shell mới sẽ được tạo ra mỗi khi quá trình thực thi chương trình bắt đầu và thường có một cửa sổ (giao diện) đi kèm với shell đó.

Chúng tôi khuyên bạn nên khởi động một Python shell ngay bây giờ và sử dụng nó để chạy thử các ví dụ trong phần còn lại của chương trình này, cũng như cho các phần tiếp theo của cuốn sách.

Một lệnh, thường được gọi là một câu lệnh (statement), chỉ thị cho trình thông dịch thực hiện một việc gì đó. Ví dụ, câu lệnh print('Yankees rule!') chỉ thị cho trình thông dịch gọi hàm[7] print, hàm này sẽ xuất chuỗi Yankees rule! ra cửa sổ liên kết với shell.

Chuỗi các lệnh sau:

```
print('Yankees rule!')
print('But not in Boston!')
print('Yankees rule,', 'but not in Boston!')
```

sẽ khiến trình thông dịch tạo ra kết quả đầu ra:

```
Yankees rule!
But not in Boston!
Yankees rule, but not in Boston!
```

Lưu ý rằng có hai giá trị đã được truyền vào hàm print trong câu lệnh thứ ba. Hàm print có thể nhận một số lượng đối số (arguments) thay đổi, ngăn cách nhau bởi dấu phẩy, và in chúng ra theo thứ tự xuất hiện, cách nhau bởi một ký tự dấu cách.

2.1.1 Đối tượng, Biểu thức và các Kiểu số học

Đối tượng (objects) là những thực thể cốt lõi mà các chương trình Python thao tác. Mỗi đối tượng đều có một kiểu (type) nhằm xác định những loại công việc mà chương trình có thể thực hiện với đối tượng đó. Các kiểu dữ liệu có thể là vô hướng (scalar) hoặc phi vô hướng (non-scalar). Các đối tượng vô hướng là không thể chia nhỏ. Hãy coi chúng như những "nguyên tử" của ngôn ngữ.[9] Ngược lại, các đối tượng phi vô hướng, ví dụ như chuỗi ký tự (strings), có cấu trúc nội bộ bên trong.

Nhiều loại đối tượng có thể được biểu diễn bằng các giá trị trực tiếp (literals) ngay trong mã nguồn của chương trình. Ví dụ, văn bản 2 là một giá trị trực tiếp biểu diễn một con số và văn bản 'abc' là một giá trị trực tiếp biểu diễn một chuỗi.

Python có bốn kiểu đối tượng vô hướng:

- **int:** được dùng để biểu diễn các số nguyên. Các giá trị trực tiếp kiểu int được viết theo cách chúng ta thường ký hiệu số nguyên (ví dụ: -3, 5 hoặc 10002).
- **float:** được dùng để biểu diễn số thực. Các giá trị trực tiếp kiểu float luôn bao gồm một dấu chấm thập phân (ví dụ: 3.0, 3.17 hoặc -28.72). (Cũng có thể viết các giá trị trực tiếp kiểu float bằng ký hiệu khoa học. Ví dụ, giá trị trực tiếp 1.6E3 đại diện cho 1.6×10^3, tức là nó tương đương với 1600.0). Bạn có thể thắc mắc tại sao kiểu này không được gọi là real (số thực). Trong máy tính, các giá trị kiểu float được lưu trữ dưới dạng số dấu phẩy động (floating point numbers). Cách biểu diễn này, vốn được sử dụng bởi tất cả các ngôn ngữ lập trình hiện đại, có nhiều ưu điểm. Tuy nhiên, trong một số tình huống, nó khiến các phép toán dấu phẩy động có hành vi hơi khác so với toán học trên số thực thông thường. Chúng ta sẽ thảo luận vấn đề này trong Mục 3.4.
- **bool:** được dùng để biểu diễn các giá trị Boolean là True (Đúng) và False (Sai).
- **None:** là một kiểu dữ liệu chỉ có một giá trị duy nhất. Chúng ta sẽ nói rõ hơn về điều này trong Mục 4.1.

Các đối tượng và toán tử có thể được kết hợp để tạo thành các biểu thức (expressions), mỗi biểu thức sẽ được tính toán để cho ra một đối tượng thuộc một kiểu nào đó. Chúng ta sẽ gọi đây là giá trị của biểu thức. Ví dụ, biểu thức 3 + 2 biểu thị đối tượng 5 kiểu int, và biểu thức 3.0 + 2.0 biểu thị đối tượng 5.0 kiểu float.

Toán tử == được dùng để kiểm tra xem hai biểu thức có tính toán ra cùng một giá trị hay không, và toán tử != được dùng để kiểm tra xem hai biểu thức có cho ra các giá trị khác nhau hay không. Một dấu

bằng đơn = mang ý nghĩa hoàn toàn khác, như chúng ta sẽ thấy ở Mục 2.1.2. Hãy lưu ý trước: bạn sẽ rất hay mắc lỗi gõ = khi thực ra ý bạn là ==. Hãy luôn để mắt tới lỗi này.

Ký hiệu »> là một dấu nhắc lệnh (shell prompt) cho biết trình thông dịch đang đợi người dùng nhập mã Python vào shell. Dòng bên dưới dòng có dấu nhắc lệnh được tạo ra khi trình thông dịch tính toán mã Python đã nhập, như được minh họa qua tương tác sau:

```
>>> 3 + 2
5
>>> 3.0 + 2.0
5.0
>>> 3 != 2
True
```

Hàm tích hợp type() của Python có thể được dùng để tìm ra kiểu của một đối tượng:

```
>>> type(3)
<class 'int'>
>>> type(3.0)
<class 'float'>
```

Các toán tử trên đối tượng kiểu int và float được liệt kê trong Hình 2.1.

Hình 2.1: Các toán tử trên kiểu dữ liệu int và float

- `i + j`: là tổng của *i* và *j*. Nếu cả *i* và *j* đều thuộc kiểu int, kết quả là một số nguyên (int). Nếu một trong hai là float, kết quả là một số thực (float).
- `i - j`: là *i* trừ *j*. Nếu cả *i* và *j* đều thuộc kiểu int, kết quả là một số nguyên (int). Nếu một trong hai là float, kết quả là một số thực (float).
- `i * j`: là tích của *i* và *j*. Nếu cả *i* và *j* đều thuộc kiểu int, kết quả là một số nguyên (int). Nếu một trong hai là float, kết quả là một số thực (float).
- `i // j`: là phép chia nguyên (integer division). Ví dụ: giá trị của `6 // 2` là số nguyên 3 và giá trị của `6 // 4` là số nguyên 1. Kết quả là 1 vì phép chia nguyên chỉ trả về thương số và bỏ qua phần dư. Nếu $j == 0$, một lỗi sẽ xảy ra.
- `i / j`: là *i* chia cho *j*. Trong Python 3, toán tử / thực hiện phép chia dấu phẩy động (floating point division). Ví dụ: giá trị của `6 / 4` là 1.5. Nếu $j == 0$, một lỗi sẽ xảy ra. (Trong Python 2, khi cả *i* và *j* đều thuộc kiểu int, toán tử / hoạt động giống như // và trả về một số nguyên. Nếu *i* hoặc *j* là float, nó hoạt động giống như toán tử / của Python 3).
- `i % j`: là phần dư khi số nguyên *i* chia cho số nguyên *j*. Nó thường được đọc là "i mod j", viết tắt của "i modulo j".
- `i ** j`: là *i* lũy thừa *j* (i^j). Nếu cả *i* và *j* đều thuộc kiểu int, kết quả là một số nguyên (int). Nếu một trong hai là float, kết quả là một số thực (float).
- **Các toán tử so sánh:** bao gồm == (bằng), != (khác/không bằng), > (lớn hơn), >= (lớn hơn hoặc bằng/tối thiểu), < (nhỏ hơn) và <= (nhỏ hơn hoặc bằng/tối đa).

Các toán tử số học có thứ tự ưu tiên thông thường. Ví dụ, toán tử * có độ ưu tiên cao hơn (liên kết chặt chẽ hơn) so với +, vì vậy biểu thức $x + y * 2$ được tính toán bằng cách nhân y với 2 trước, sau đó mới cộng kết quả với x. Thứ tự tính toán có thể được thay đổi bằng cách sử dụng dấu ngoặc đơn để nhóm các biểu thức con, ví dụ: $(x + y) * 2$ sẽ thực hiện phép cộng x và y trước, sau đó mới nhân kết quả với 2.

Các toán tử sơ khai trên kiểu bool:

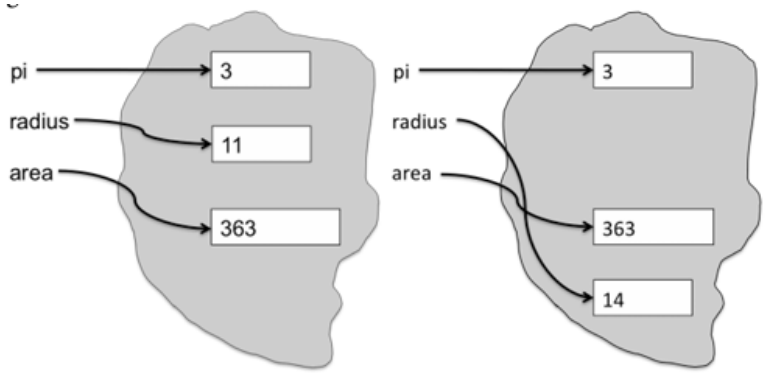
- *a* `and` *b*: Trả về True nếu cả *a* và *b* đều là True, ngược lại trả về False.
- *a* `or` *b*: Trả về True nếu ít nhất một trong hai giá trị *a* hoặc *b* là True, ngược lại trả về False.
- `not` *a*: Trả về True nếu *a* là False, và trả về False nếu *a* là True.

2.1.2 Biến và Phép gán

Biến cung cấp một cách để gắn tên với các đối tượng. Hãy xem xét đoạn mã sau:

```
pi = 3
radius = 11
area = pi * (radius**2)
radius = 14
```

Đầu tiên, chương trình liên kết các tên pi và radius với các đối tượng khác nhau thuộc kiểu int. Sau đó, nó liên kết tên area với một đối tượng thứ ba cũng thuộc kiểu int. Điều này được mô tả trong bảng bên trái của Hình 2.2.



Hình 2.2. Ràng buộc biến với đối tượng

Nếu chương trình tiếp tục thực thi lệnh `radius = 14`, tên radius sẽ được tái liên kết (rebound) với một đối tượng kiểu int khác, như được hiển thị ở bảng bên phải của Hình 2.2. Lưu ý rằng phép gán này không có tác động gì đến giá trị mà area đang liên kết. Biến area vẫn được liên kết với đối tượng được tạo ra từ biểu thức $3 \times (11^2)$.

Trong Python, một biến chỉ đơn thuần là một cái tên, không hơn không kém. Hãy ghi nhớ điều này— nó rất quan trọng. Một câu lệnh gán sẽ liên kết cái tên ở bên trái ký hiệu = với đối tượng được biểu diễn bởi biểu thức ở bên phải ký hiệu =. Cũng hãy ghi nhớ điều này: một đối tượng có thể có một tên, nhiều tên, hoặc không có cái tên nào liên kết với nó cả.

Có lẽ chúng ta không nên nói "một biến chỉ là một cái tên". Bất kể những gì Juliet đã nói[11], tên gọi thực sự quan trọng. Ngôn ngữ lập trình cho phép chúng ta mô tả các tính toán theo cách mà máy móc có thể thực thi. Điều này không có nghĩa là chỉ có máy tính mới đọc chương trình.

Như bạn sẽ sớm khám phá ra, việc viết các chương trình hoạt động chính xác không phải lúc nào cũng dễ dàng. Các lập trình viên kinh nghiệm sẽ xác nhận rằng họ dành rất nhiều thời gian để đọc các chương trình nhằm cố gắng hiểu tại sao chúng lại hành xử như vậy. Do đó, việc viết chương trình sao cho dễ đọc là điều tối quan trọng. Việc lựa chọn tên biến phù hợp đóng vai trò lớn trong việc tăng cường khả năng đọc hiểu.

Hãy xem xét hai đoạn mã sau:

```
# Đoạn mã 1
a = 3.14159
b = 11.2
c = a*(b**2)

# Đoạn mã 2
pi = 3.14159
diameter = 11.2
area = pi*(diameter**2)
```

Đối với Python, chúng không có gì khác biệt. Khi thực thi, chúng sẽ làm cùng một việc. Tuy nhiên, đối với người đọc, chúng lại rất khác nhau. Khi đọc đoạn mã bên trái, không có lý do tiên quyết nào để nghi ngờ có điều gì đó sai sót. Nhưng chỉ cần liếc nhìn đoạn mã bên phải, chúng ta sẽ nảy sinh nghi ngờ ngay: hoặc biến lẽ ra phải tên là radius (bán kính) thay vì diameter (đường kính), hoặc diameter phải được chia cho 2.0 trong phép tính diện tích.

Trong Python, tên biến có thể chứa:

- Các chữ cái viết hoa và viết thường
- Các chữ số (nhưng không được bắt đầu bằng chữ số)
- Ký tự đặc biệt _

Tên biến trong Python có phân biệt chữ hoa chữ thường (ví dụ: Julie và julie là hai tên khác nhau).

Cuối cùng, có một số lượng nhỏ các từ dự phòng (keywords) trong Python đã có sẵn ý nghĩa và không thể dùng làm tên biến. Các từ khóa trong Python 3 bao gồm:

- `and, as, assert, break, class, continue, def, del`
- `elif, else, except, False, finally, for, from, global`
- `if, import, in, is, lambda, nonlocal, None, not`
- `or, pass, raise, return, True, try, while, with, yield`

Một cách tốt khác để tăng khả năng đọc hiểu mã nguồn là thêm vào các chú thích (comments). Văn bản đứng sau ký hiệu # sẽ không được Python thông dịch. Ví dụ:

```
side = 1 # Chiều dài cạnh của một hình vuông đơn vị
radius = 1 # Bán kính của một hình tròn đơn vị
# Lấy diện tích hình vuông trừ đi diện tích hình tròn
areaC = pi * radius**2
areaS = side * side
difference = areaS - areaC
```

Python cho phép gán đa luồng (multiple assignment). Câu lệnh:

```
x, y = 2, 3
```

sẽ liên kết x với 2 và y với 3. Tất cả các biểu thức ở phía bên phải của phép gán đều được tính toán trước khi bất kỳ liên kết nào được thay đổi.

Điều này rất tiện lợi vì nó cho phép bạn sử dụng phép gán đa luồng để hoán đổi giá trị liên kết của hai biến. Ví dụ:

```
x, y = 2, 3
x, y = y, x
print('x =', x)
print('y =', y)
```

đoạn mã trên sẽ in ra:

```
x = 3
y = 2
```

2.1.3 Các trình thông dịch (IDE) của Python

Việc nhập chương trình trực tiếp vào shell là cực kỳ bất tiện. Hầu hết các lập trình viên thích sử dụng một loại trình soạn thảo văn bản nào đó vốn là một phần của Môi trường Phát triển Tích hợp (Integrated Development Environment - IDE).

Một IDE có tên là IDLE được đi kèm sẵn trong gói cài đặt chuẩn của Python. Khi Python trở nên phổ biến hơn, các IDE khác cũng đã xuất hiện. Những IDE mới này thường tích hợp một số thư viện Python phổ biến và cung cấp các tiện ích mà IDLE không có. Anaconda và Canopy là hai trong số những IDE phổ biến nhất hiện nay. Các mã nguồn xuất hiện trong cuốn sách này được tạo và kiểm thử bằng Anaconda.

IDE là các ứng dụng, giống như bất kỳ ứng dụng nào khác trên máy tính của bạn. Hãy khởi động chúng theo cách bạn vẫn làm với các ứng dụng khác, ví dụ như nhấp đúp vào một biểu tượng.

Tất cả các IDE của Python đều cung cấp:

- Một trình soạn thảo văn bản với các tính năng:
  - Làm nổi bật cú pháp (syntax highlighting)

  - Tự động hoàn thành mã (auto completion)
  - Thụt lề thông minh (smart indentation)

- Một shell có tính năng làm nổi bật cú pháp
- Một trình gỡ lỗi tích hợp (integrated debugger), thứ mà hiện tại bạn có thể tạm thời bỏ qua

Khi IDE khởi động, nó sẽ mở một cửa sổ shell để bạn có thể nhập các lệnh Python. Nó cũng cung cấp cho bạn một menu tệp (File) và một menu chỉnh sửa (Edit), cùng với một số menu khác giúp việc thực hiện các thao tác như in chương trình trở nên thuận tiện hơn.

Menu tệp (File menu) bao gồm các lệnh để:

- Tạo một cửa sổ soạn thảo mới để bạn nhập chương trình Python
- Mở một tệp chứa chương trình Python hiện có
- Lưu nội dung của cửa sổ soạn thảo hiện tại vào một tệp (với phần mở rộng là `.py`)

Menu chỉnh sửa (Edit menu) bao gồm các lệnh soạn thảo văn bản tiêu chuẩn:

- Các lệnh cơ bản như sao chép (copy), dán (paste) và tìm kiếm (find)
- Các lệnh được thiết kế riêng để chỉnh sửa mã Python dễ dàng hơn, chẳng hạn như:
  - Thụt lề vùng chọn (indent region)
  - Chú thích hóa vùng chọn (comment out region)

Để biết thêm thông tin về một số IDE phổ biến, hãy truy cập:

- `http://docs.python.org/library/idle.html/`
- `https://store.continuum.io/cshop/anaconda/`
- `https://www.enthought.com/products/canopy/`

2.2 Chương trình rẽ nhánh

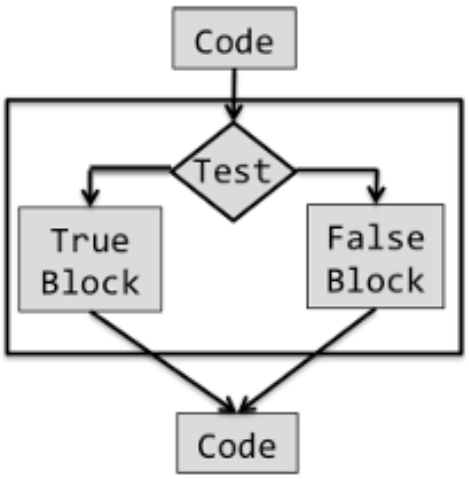
Các loại tính toán mà chúng ta đã xem xét cho đến nay được gọi là chương trình tuyến tính (straight-line programs). Chúng thực thi hết câu lệnh này đến câu lệnh khác theo thứ tự xuất hiện và dừng lại khi không còn câu lệnh nào nữa. Những loại tính toán mà chúng ta có thể mô tả bằng chương trình tuyến tính thường không mấy thú vị; thực tế, chúng khá là nhàm chán.

Các chương trình rẽ nhánh (branching programs) thì thú vị hơn nhiều. Câu lệnh rẽ nhánh đơn giản nhất là câu lệnh điều kiện (conditional). Như được trình bày trong phần đóng khung của Hình 2.3, một câu lệnh điều kiện gồm ba phần:

- Một phép thử (test): tức là một biểu thức có giá trị tính toán là True (Đúng) hoặc False (Sai)

- Một khối mã (block of code): sẽ được thực thi nếu phép thử cho kết quả là True
- Một khối mã tùy chọn (optional block of code): sẽ được thực thi nếu phép thử cho kết quả là False

Sau khi một câu lệnh điều kiện được thực thi, việc thực thi chương trình sẽ tiếp tục tại phần mã nằm sau câu lệnh đó.



Hình 2.3: Lưu đồ cho câu lệnh điều kiện

Trong Python, một câu lệnh điều kiện có dạng:

```
if biểu thức Boolean:
    khối mã
else:
    khối mã
```

hoặc

```
if biểu thức Boolean:
    khối mã
```

Khi mô tả dạng thức của các câu lệnh Python, chúng tôi sử dụng chữ in nghiêng để mô tả các loại mã có thể xuất hiện tại vị trí đó trong chương trình. Ví dụ, biểu thức Boolean chỉ ra rằng bất kỳ biểu thức nào có giá trị True hoặc False đều có thể đứng sau từ khóa `if`, và khối mã chỉ ra rằng bất kỳ chuỗi câu lệnh Python nào cũng có thể đứng sau `else:`.

Hãy xem xét chương trình sau đây, nó sẽ in ra ”Even” (Chẵn) nếu giá trị của biến x là số chẵn và in ”Odd” (Lẻ) trong trường hợp ngược lại:

```
if x % 2 == 0:
    print('Even')
else:
    print('Odd')
print('Done with conditional')
```

Biểu thức `x % 2 == 0` sẽ trả về True khi phần dư của x chia cho 2 bằng 0, và trả về False trong các trường hợp còn lại. Hãy nhớ rằng == được dùng để so sánh, vì dấu = đã được dành riêng cho phép gán.

Việc thụt lề (indentation) có ý nghĩa quan trọng về mặt ngữ nghĩa trong Python. Ví dụ, nếu câu lệnh cuối cùng trong đoạn mã trên được thụt lề, nó sẽ trở thành một phần của khối mã liên kết với `else`, thay vì là khối mã nằm sau câu lệnh điều kiện.

Python khá khác biệt khi sử dụng việc thụt lề theo cách này. Hầu hết các ngôn ngữ lập trình khác sử dụng một số loại ký hiệu đóng ngoặc để phân định các khối mã, ví dụ: C bao bọc các khối mã trong dấu ngoặc nhọn { }. Một ưu điểm của cách tiếp cận trong Python là nó đảm bảo cấu trúc trực quan của chương trình là một sự thể hiện chính xác cấu trúc ngữ nghĩa của chương trình đó. Vì thụt lề có ý nghĩa quan trọng về mặt ngữ nghĩa, nên khái niệm về một "dòng mã" là rất quan trọng.

Khi khối đúng (true block) hoặc khối sai (false block) của một câu lệnh điều kiện chứa một câu lệnh điều kiện khác, các câu lệnh điều kiện đó được gọi là lồng nhau (nested). Trong đoạn mã dưới đây, có các câu lệnh điều kiện lồng nhau ở cả hai nhánh của câu lệnh if cấp cao nhất:

```
if x % 2 == 0:
    if x % 3 == 0:
        print('Divisible by 2 and 3')
    else:
        print('Divisible by 2 and not by 3')
elif x % 3 == 0:
    print('Divisible by 3 and not by 2')
```

Từ khóa `elif` trong đoạn mã trên là viết tắt của "else if".

Việc sử dụng một biểu thức Boolean phức hợp trong phép thử của một câu lệnh điều kiện thường rất tiện lợi, ví dụ:

```
if x < y and x < z:
    print('x is least')
elif y < z:
    print('y is least')
else:
    print('z is least')
```

Câu lệnh điều kiện cho phép chúng ta viết các chương trình thú vị hơn chương trình tuyến tính, nhưng lớp các chương trình rẽ nhánh vẫn còn khá hạn chế. Một cách để tư duy về sức mạnh của một lớp chương trình là dựa trên thời gian thực thi của chúng. Giả sử mỗi dòng mã mất một đơn vị thời gian để thực thi. Nếu một chương trình tuyến tính có n dòng mã, nó sẽ mất n đơn vị thời gian để chạy. Vậy còn chương trình rẽ nhánh với n dòng mã thì sao? Nó có thể mất ít hơn n đơn vị thời gian, nhưng không thể nhiều hơn, vì mỗi dòng mã được thực thi tối đa một lần.

Một chương trình mà thời gian chạy tối đa bị giới hạn bởi độ dài của chương trình được gọi là chạy trong thời gian hằng số (constant time). Điều này không có nghĩa là mỗi lần chạy nó đều thực hiện cùng một số bước. Nó có nghĩa là tồn tại một hằng số k, sao cho chương trình được đảm bảo không tốn quá k bước để chạy. Điều này ngụ ý rằng thời gian chạy không tăng theo kích thước của dữ liệu đầu vào.

Các chương trình thời gian hằng số khá hạn chế trong những gì chúng có thể làm. Hãy xem xét việc viết một chương trình để kiểm phiếu trong một cuộc bầu cử. Sẽ thật đáng ngạc nhiên nếu ai đó có thể

viết một chương trình làm việc này trong một khoảng thời gian độc lập với số lượng phiếu bầu. Thực tế, người ta có thể chứng minh rằng việc đó là bất khả thi. Việc nghiên cứu về độ khó nội tại của các bài toán là chủ đề của độ phức tạp tính toán (computational complexity). Chúng ta sẽ quay lại chủ đề này vài lần trong cuốn sách.

May mắn thay, chúng ta chỉ cần thêm một cấu trúc ngôn ngữ lập trình nữa, đó là vòng lặp (iteration), để có thể viết các chương trình có độ phức tạp tùy ý. Chúng ta sẽ tìm hiểu điều đó trong Mục 2.4.

Bài tập nhỏ (Finger exercise): Hãy viết một chương trình kiểm tra ba biến—x, y, và z—và in ra số lẻ lớn nhất trong số chúng. Nếu không có số nào là số lẻ, chương trình nên in ra một thông báo tương ứng.

2.3 Chuỗi ký tự và Nhập liệu

Các đối tượng thuộc kiểu `str` được dùng để biểu diễn các chuỗi ký tự.[13] Các giá trị trực tiếp (literals) kiểu `str` có thể được viết bằng dấu ngoặc đơn hoặc ngoặc kép, ví dụ: `'abc'` hoặc `"abc"`. Giá trị trực tiếp `'123'` biểu thị một chuỗi gồm ba ký tự, không phải là con số một trăm hai mươi ba.

Hãy thử nhập các biểu thức sau vào trình thông dịch Python (nhớ rằng >>> là dấu nhắc lệnh, không phải nội dung bạn cần nhập):

```
>>> 'a'
>>> 3*4
>>> 3*'a'
>>> 3+4
>>> 'a'+'a'
```

Toán tử + được gọi là nạp chồng (overloaded): Nó có các ý nghĩa khác nhau tùy thuộc vào kiểu của các đối tượng mà nó được áp dụng. Ví dụ, nó có nghĩa là phép cộng khi áp dụng cho hai con số và là phép nối (concatenation) khi áp dụng cho hai chuỗi. Toán tử $*$ cũng được nạp chồng. Nó mang ý nghĩa như bạn mong đợi khi cả hai toán hạng đều là số. Khi áp dụng cho một số nguyên (`int`) và một chuỗi (`str`), nó là một toán tử lặp (repetition) — biểu thức $n * s$, trong đó n là một số nguyên và s là một chuỗi, sẽ tính toán ra một chuỗi gồm n lần lặp lại của s. Ví dụ, biểu thức `2*'John'` có giá trị là `'JohnJohn'`. Điều này hoàn toàn có logic. Giống như biểu thức toán học 3×2 tương đương với $2 + 2 + 2$, biểu thức `3*'a'` tương đương với `'a'+'a'+'a'`.

Bây giờ hãy thử nhập:

```
>>> a
>>> 'a'*'a'
```

Mỗi dòng này đều tạo ra một thông báo lỗi. Dòng đầu tiên tạo ra thông báo:

```
NameError: name 'a' is not defined
```

Vì `a` không phải là giá trị trực tiếp của bất kỳ kiểu dữ liệu nào, trình thông dịch coi nó như một cái tên (biến). Tuy nhiên, vì cái tên đó chưa được liên kết với bất kỳ đối tượng nào, việc cố gắng sử dụng nó sẽ gây ra lỗi thực thi (runtime error).

Mã `'a'*'a'` tạo ra thông báo lỗi:

```
TypeError: can't multiply sequence by non-int of type 'str'
```

Việc tồn tại cơ chế kiểm tra kiểu (type checking) là một điều tốt. Nó biến những sai sót vô ý (và đôi khi tinh vi) thành các lỗi làm dừng việc thực thi, thay vì những lỗi khiến chương trình hành xử theo những cách bí ẩn. Việc kiểm tra kiểu trong Python không chặt chẽ như một số ngôn ngữ lập trình khác (ví dụ: Java), nhưng nó tốt hơn trong Python 3 so với Python 2. Ví dụ, khá rõ ràng về ý nghĩa của dấu < khi nó được dùng để so sánh hai chuỗi hoặc hai con số. Nhưng giá trị của `'4' < 3` nên là gì? Một cách khá tùy tiện, những nhà thiết kế Python 2 đã quyết định nó nên là `False`, vì tất cả các giá trị số đều nhỏ hơn tất cả các giá trị thuộc kiểu `str`. Những nhà thiết kế Python 3 và hầu hết các ngôn ngữ hiện đại khác đã quyết định rằng vì những biểu thức như vậy không có ý nghĩa rõ ràng, chúng nên tạo ra một thông báo lỗi.

Chuỗi là một trong số các kiểu tuần tự (sequence types) trong Python. Chúng chia sẻ các thao tác sau với tất cả các kiểu tuần tự khác:

- Độ dài của một chuỗi có thể được tìm thấy bằng hàm `len`. Ví dụ, giá trị của `len('abc')` là 3.
- Truy xuất chỉ số (Indexing) có thể được dùng để trích xuất các ký tự riêng lẻ từ một chuỗi. Trong Python, tất cả việc đánh chỉ số đều bắt đầu từ số 0 (zero-based). Ví dụ, nhập `'abc'[0]` vào trình thông dịch sẽ khiến nó hiển thị chuỗi `'a'`. Nhập `'abc'[3]` sẽ tạo ra thông báo lỗi `IndexError: string index out of range`. Vì Python sử dụng 0 để chỉ phần tử đầu tiên của chuỗi, phần tử cuối cùng của một chuỗi có độ dài bằng 3 được truy cập bằng chỉ số 2. Các số âm được dùng để đánh chỉ số tính từ cuối chuỗi. Ví dụ, giá trị của `'abc'[-1]` là `'c'`.
- Cắt chuỗi (Slicing) được dùng để trích xuất các chuỗi con có độ dài tùy ý. Nếu `s` là một chuỗi, biểu thức `s[start:end]` biểu thị chuỗi con của `s` bắt đầu tại chỉ số `start` và kết thúc tại chỉ số `end-1`. Ví dụ, `'abc'[1:3] == 'bc'`. Tại sao nó lại kết thúc ở chỉ số `end-1` thay vì `end`? Để những biểu thức như `'abc'[0:len('abc')]` có giá trị đúng như mong đợi. Nếu giá trị trước dấu hai chấm bị bỏ trống, nó mặc định là 0. Nếu giá trị sau dấu hai chấm bị bỏ trống, nó mặc định là độ dài của chuỗi. Do đó, biểu thức `'abc'[:]` có ngữ nghĩa tương đương với biểu thức dài dòng hơn là `'abc'[0:len('abc')]`.

2.3.1 Nhập liệu (Input)

Python 3 có một hàm là `input`, được dùng để nhận dữ liệu nhập trực tiếp từ người dùng.[14] Hàm này nhận một chuỗi làm đối số và hiển thị chuỗi đó như một lời nhắc (prompt) trong shell. Sau đó, nó đợi người dùng nhập nội dung và nhấn phím Enter. Dòng dữ liệu mà người dùng đã nhập sẽ được coi là một chuỗi và trở thành giá trị trả về của hàm.

Hãy xem xét đoạn mã sau:

```
>>> name = input('Enter your name: ')
Enter your name: George Washington
>>> print('Are you really', name, '?')
Are you really George Washington ?
>>> print('Are you really ' + name + '?')
Are you really George Washington?
```

Lưu ý rằng câu lệnh `print` đầu tiên đã chèn một khoảng trắng trước dấu `"?"`. Điều này xảy ra bởi vì khi hàm `print` nhận nhiều đối số, nó sẽ tự động đặt một khoảng cách giữa các giá trị liên kết với các đối số đó. Câu lệnh `print` thứ hai sử dụng phép nối chuỗi (concatenation) để tạo ra một chuỗi không chứa khoảng trắng thừa và truyền chuỗi này làm đối số duy nhất cho hàm `print`.

Bây giờ, hãy xem xét:

```
>>> n = input('Enter an int: ')
Enter an int: 3
>>> print(type(n))
```

Lưu ý rằng biến `n` được liên kết với chuỗi `'3'` chứ không phải số nguyên 3. Vì vậy, ví dụ, giá trị của biểu thức `n * 4` sẽ là `'3333'` thay vì 12. Tin tốt là bất khi nào một chuỗi là một giá trị trực tiếp (literal) hợp lệ của một kiểu dữ liệu nào đó, chúng ta đều có thể áp dụng phép chuyển đổi kiểu.

Chuyển đổi kiểu dữ liệu (còn gọi là ép kiểu - type casts) được sử dụng thường xuyên trong mã nguồn Python. Chúng ta sử dụng tên của một kiểu dữ liệu để chuyển đổi các giá trị sang kiểu đó. Ví dụ, giá trị của biểu thức `int('3') * 4` là 12. Khi một số thực (`float`) được chuyển đổi sang một số nguyên (`int`), con số đó sẽ bị cắt bỏ phần thập phân (chứ không phải làm tròn), ví dụ: giá trị của `int(3.9)` là số nguyên 3.

2.3.2 Chú giải về Mã hóa ký tự (Character Encoding)

Trong nhiều năm, hầu hết các ngôn ngữ lập trình đã sử dụng một tiêu chuẩn gọi là ASCII để biểu diễn nội bộ các ký tự. Tiêu chuẩn này bao gồm 128 ký tự, đủ để biểu diễn tập hợp các ký tự thông thường xuất hiện trong văn bản tiếng Anh — nhưng không đủ để bao quát các ký tự và các dấu phụ xuất hiện trong tất cả các ngôn ngữ trên thế giới.

Trong những năm gần đây, đã có sự chuyển dịch sang Unicode. Tiêu chuẩn Unicode là một hệ thống mã hóa ký tự được thiết kế để hỗ trợ việc xử lý kỹ thuật số và hiển thị văn bản viết của tất cả các ngôn ngữ. Tiêu chuẩn này chứa hơn 120.000 ký tự khác nhau — bao quát 129 hệ chữ viết hiện đại và lịch sử cùng nhiều bộ ký hiệu khác nhau.

Tiêu chuẩn Unicode có thể được hiện thực hóa bằng các cách mã hóa ký tự nội bộ khác nhau. Bạn có thể chỉ định cho Python biết nên sử dụng bảng mã nào bằng cách chèn một dòng chú thích theo định dạng:

```
# -*- coding: tên_bảng_mã -*-
```

vào dòng đầu tiên hoặc thứ hai của chương trình. Ví dụ:

```
# -*- coding: utf-8 -*-
```

dòng này chỉ thị cho Python sử dụng UTF-8, bảng mã ký tự được sử dụng thường xuyên nhất cho các trang World Wide Web.[15] Nếu bạn không có dòng chú thích như vậy trong chương trình, hầu hết các bản thực thi Python sẽ mặc định sử dụng UTF-8.

Khi sử dụng UTF-8, nếu trình soạn thảo văn bản cho phép, bạn có thể nhập trực tiếp các đoạn mã như:

```
print('Mluvíš anglicky?')
```

kết quả sẽ in ra:

```
Mluvíš anglicky?
```

Bạn có thể thắc mắc làm thế nào tôi có thể gõ được chuỗi `'Mluvíš anglicky?'`. Thực tế là tôi không gõ. Bởi vì hầu hết World Wide Web đều sử dụng UTF-8, tôi có thể sao chép chuỗi đó từ một trang web và dán trực tiếp vào chương trình của mình.

2.4 Vòng lặp

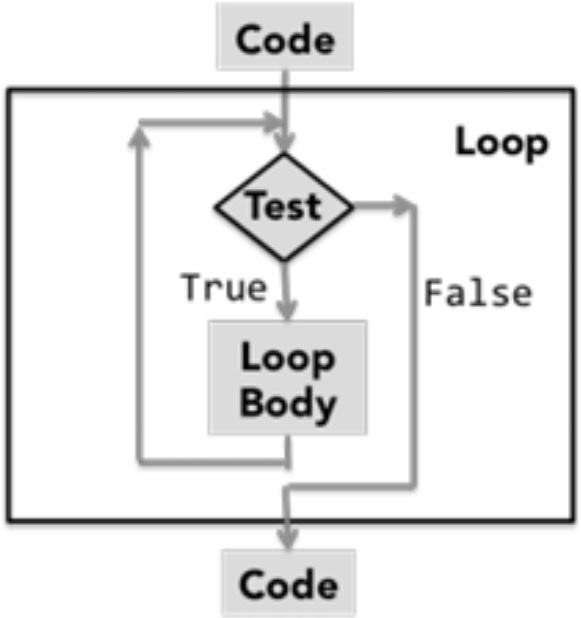
Chúng ta đã kết thúc Mục 2.2 với nhận định rằng hầu hết các tác vụ tính toán không thể hoàn thành chỉ bằng các chương trình rẽ nhánh. Hãy thử tưởng tượng việc viết một chương trình yêu cầu người dùng nhập số lần họ muốn in chữ cái "X", sau đó in ra một chuỗi với số lượng chữ X tương ứng. Chúng ta có thể nghĩ đến việc viết như sau:

```
numXs = int(input('How many times should I print the letter X? '))
toPrint = ''
if numXs == 1:
    toPrint = 'X'
elif numXs == 2:
    toPrint = 'XX'
elif numXs == 3:
    toPrint = 'XXX'
# ...
print(toPrint)
```

Nhưng sẽ nhanh chóng nhận thấy rằng chúng ta cần số lượng câu lệnh điều kiện tương đương với số lượng các số nguyên dương — và thực tế là có vô số số nguyên dương. Cái chúng ta thực sự cần là một chương trình có dạng:

```
numXs = int(input('How many times should I print the letter X? '))
toPrint = ''
# nối X vào toPrint numXs lần
print(toPrint)
```

Khi muốn một chương trình thực hiện cùng một việc nhiều lần, chúng ta có thể sử dụng vòng lặp (iteration hay looping). Cơ chế lặp tổng quát được thể hiện trong phần đóng khung của Hình 2.4. Giống như câu lệnh điều kiện, nó bắt đầu bằng một phép thử. Nếu phép thử trả về True, chương trình thực thi thân vòng lặp (loop body) một lần, rồi quay lại đánh giá lại phép thử. Quá trình này lặp lại cho đến khi phép thử trả về False, sau đó quyền điều khiển được chuyển đến đoạn mã nằm sau câu lệnh lặp.



Hình 2.4: Lưu đồ cho vòng lặp

Chúng ta có thể viết loại vòng lặp được mô tả trong Hình 2.4 bằng câu lệnh `while`. Hãy xem ví dụ sau:

```
# Tính bình phương một số nguyên bằng phương pháp cộng lặp
x = 3
ans = 0
itersLeft = x
while (itersLeft != 0):
    ans = ans + x
    itersLeft = itersLeft - 1
print(str(x) + '*' + str(x) + ' = ' + str(ans))
```

Mã nguồn bắt đầu bằng cách liên kết biến x với số nguyên 3. Sau đó, nó thực hiện bình phương x bằng cách cộng lặp lại. Bảng trong Hình 2.5 cho thấy giá trị gắn với mỗi biến mỗi khi chạm đến phép thử ở đầu vòng lặp. Chúng tôi xây dựng bảng này bằng cách mô phỏng thủ công (hand-simulating) đoạn mã, tức là đóng vai một trình thông dịch Python và thực thi chương trình bằng bút và giấy. Sử dụng bút và giấy nghe có vẻ cổ điển, nhưng đó là một cách tuyệt vời để hiểu cách một chương trình vận hành.

Hình 2.5: Mô phỏng thủ công một chương trình nhỏ

Lần thử (Test #)	**x**	**ans**	**itersLeft**
1	3	0	3
2	3	3	2
3	3	6	1
4	3	9	0

Lần thứ tư chạm đến phép thử, nó trả về False và luồng điều khiển chuyển đến câu lệnh `print` nằm

sau vòng lặp. Với những giá trị nào của x thì chương trình này sẽ kết thúc? Có ba trường hợp cần xem xét: $x = 0, x > 0$, và $x < 0$.

- Nếu $x = 0$: Giá trị ban đầu của `itersLeft` cũng sẽ là 0, và thân vòng lặp sẽ không bao giờ được thực thi.
- Nếu $x > 0$: Giá trị ban đầu của `itersLeft` sẽ lớn hơn 0, và thân vòng lặp sẽ được thực thi ít nhất một lần. Mỗi lần thực thi thân vòng lặp, giá trị của `itersLeft` giảm đi đúng 1 đơn vị. Điều này có nghĩa là nếu `itersLeft` bắt đầu lớn hơn 0, sau một số lần lặp hữu hạn, `itersLeft` sẽ bằng 0. Tại thời điểm đó, phép thử vòng lặp trả về False và quyền điều khiển chuyển đến mã sau câu lệnh `while`.
- Nếu $x < 0$: Một điều rất tệ sẽ xảy ra. Quyền điều khiển sẽ đi vào vòng lặp, và mỗi lần lặp lại khiến `itersLeft` càng xa số 0 thay vì lại gần nó hơn. Chương trình sẽ tiếp tục thực hiện vòng lặp mãi mãi (vòng lặp vô hạn). Làm thế nào để khắc phục lỗi này? Việc khởi tạo `itersLeft` bằng giá trị tuyệt đối của x (`abs(x)`) gần như giải quyết được vấn đề. Vòng lặp sẽ kết thúc, nhưng nó sẽ in ra một giá trị âm. Nếu câu lệnh gán bên trong vòng lặp cũng được đổi thành `ans = ans + abs(x)`, mã nguồn sẽ hoạt động chính xác.

Bài tập nhỏ: Hãy thay thế phần chú thích trong đoạn mã sau bằng một vòng lặp `while`.

```
numXs = int(input('How many times should I print the letter X? '))
toPrint = ''
# nối X vào toPrint numXs lần
print(toPrint)
```

Đôi khi việc thoát khỏi vòng lặp mà không cần kiểm tra điều kiện lặp là rất tiện lợi. Việc thực thi câu lệnh `break` sẽ chấm dứt vòng lặp chứa nó và chuyển quyền điều khiển đến đoạn mã ngay sau vòng lặp đó. Ví dụ:

```
# Tìm một số nguyên dương chia hết cho cả 11 và 12
x = 1
while True:
    if x%11 == 0 and x%12 == 0:
        break
    x = x + 1
print(x, 'is divisible by 11 and 12')
```

Kết quả in ra: `132 is divisible by 11 and 12`. Nếu một câu lệnh `break` được thực thi bên trong một vòng lặp lồng nhau (vòng lặp nằm trong vòng lặp khác), lệnh `break` sẽ chỉ chấm dứt vòng lặp bên trong cùng chứa nó.

Đến đây, chúng ta đã bao quát hầu hết mọi thứ về Python cần thiết để bắt đầu viết các chương trình thú vị xử lý số và chuỗi. Trong chương tiếp theo, chúng ta sẽ tạm nghỉ việc học Python để sử dụng những gì đã biết giải quyết một số bài toán đơn giản.

Bài tập nhỏ: Viết một chương trình yêu cầu người dùng nhập vào 10 số nguyên, sau đó in ra số lẻ lớn nhất trong số các số đã nhập. Nếu không có số lẻ nào được nhập, chương trình nên in ra một thông báo tương ứng.

3. MỘT SỐ CHƯƠNG TRÌNH SỐ HỌC ĐƠN GIẢN

Bây giờ, sau khi đã tìm hiểu về một số cấu trúc cơ bản của Python, đã đến lúc chúng ta bắt đầu suy nghĩ về cách kết hợp những cấu trúc đó để viết các chương trình đơn giản. Trong quá trình thực hiện, chúng tôi sẽ lồng ghép thêm một vài cấu trúc ngôn ngữ và một số kỹ thuật thuật toán khác.

3.1 Liệt kê vét cạn (Exhaustive Enumeration)

Đoạn mã trong Hình 3.1 in ra căn bậc ba số nguyên của một số nguyên (nếu nó tồn tại). Nếu giá trị nhập vào không phải là một số lập phương hoàn chỉnh, chương trình sẽ in ra một thông báo tương ứng.

Hình 3.1: Sử dụng phương pháp duyệt cạn kiệt để tìm căn bậc ba

```
# Tìm căn bậc ba của một số lập phương hoàn chỉnh
x = int(input('Enter an integer: '))
ans = 0
while ans**3 < abs(x):
    ans = ans + 1
if ans**3 != abs(x):
    print(x, 'is not a perfect cube')
else:
    if x < 0:
        ans = -ans
    print('Cube root of', x, 'is', ans)
```

Với những giá trị nào của x thì chương trình này sẽ kết thúc? Câu trả lời là: “Mọi số nguyên”. Điều này có thể được lập luận khá đơn giản:

- Giá trị của biểu thức `ans**3` bắt đầu từ 0 và lớn dần sau mỗi lần lặp.
- Khi nó đạt đến hoặc vượt quá `abs(x)`, vòng lặp sẽ kết thúc.
- Vì `abs(x)` luôn là số dương, nên chỉ có một số lượng hữu hạn các lần lặp trước khi vòng lặp buộc phải kết thúc.

Mỗi khi viết một vòng lặp, bạn nên nghĩ về một hàm giảm (decrementing function) phù hợp. Đây là một hàm có các đặc tính sau:

- Nó ánh xạ một tập hợp các biến chương trình thành một số nguyên.
- Khi bắt đầu vào vòng lặp, giá trị của nó là không âm.
- Khi giá trị của nó ≤ 0, vòng lặp kết thúc.
- Giá trị của nó giảm đi sau mỗi lần thực hiện vòng lặp.

Hàm giảm cho vòng lặp `while` trong Hình 3.1 là gì? Đó chính là: $|x| - ans^3$.

Bây giờ, hãy thử đưa vào một số lỗi và xem điều gì xảy ra. Đầu tiên, hãy thử vô hiệu hóa (comment out) câu lệnh `ans = 0`. Trình thông dịch Python sẽ in ra thông báo lỗi:

```
NameError: name 'ans' is not defined
```

vì trình thông dịch cố gắng tìm giá trị mà `ans` liên kết trước khi nó được liên kết với bất kỳ thứ gì.

Bây giờ, hãy khôi phục việc khởi tạo `ans`, thay thế câu lệnh `ans = ans + 1` bằng `ans = ans`, và thử tìm căn bậc ba của 8. Sau khi bạn đã mệt mỏi vì chờ đợi, hãy nhấn tổ hợp phím "Control C" (nhấn giữ phím Ctrl và phím C đồng thời). Thao tác này sẽ đưa bạn quay trở lại dấu nhắc lệnh trong shell.

Tiếp theo, hãy thêm câu lệnh:

```
print('Value of the decrementing function abs(x) - ans**3 is', abs(x) - ans**3)
```

vào đầu vòng lặp và thử chạy lại. Lần này, nó sẽ in ra dòng chữ:

```
Value of the decrementing function abs(x) - ans**3 is 8
```

lặp đi lặp lại liên tục.

Chương trình sẽ chạy mãi mãi vì thân vòng lặp không còn làm giảm khoảng cách giữa ans^3 và $|x|$. Khi đối mặt với một chương trình dường như không bao giờ kết thúc, các lập trình viên kinh nghiệm thường chèn các câu lệnh `print` như ở đây để kiểm tra xem hàm giảm có thực sự đang giảm đi hay không.

Kỹ thuật thuật toán được sử dụng trong chương trình này là một biến thể của phương pháp "dự đoán và kiểm tra" được gọi là duyệt cạn kiệt (exhaustive enumeration). Chúng ta liệt kê tất cả các khả năng cho đến khi tìm thấy đáp án đúng hoặc cạn kiệt không gian các khả năng. Thoạt nhìn, đây có vẻ là một cách giải quyết vấn đề cực kỳ ngớ ngẩn. Tuy nhiên, điều đáng ngạc nhiên là các thuật toán duyệt cạn kiệt thường là cách thực tế nhất để giải quyết một bài toán. Chúng thường dễ cài đặt và dễ hiểu. Và trong nhiều trường hợp, chúng chạy đủ nhanh cho mọi mục đích thực tế.

Hãy đảm bảo bạn đã xóa hoặc vô hiệu hóa câu lệnh `print` vừa chèn vào và khôi phục lại câu lệnh `ans = ans + 1`, sau đó thử tìm căn bậc ba của 1957816251. Chương trình sẽ hoàn thành gần như ngay lập tức. Bây giờ, hãy thử với số 7406961012236344616.

Như bạn có thể thấy, ngay cả khi cần hàng triệu lần dự đoán, đó thường không phải là vấn đề. Máy tính hiện đại có tốc độ kinh ngạc. Phải mất khoảng một nano giây (một phần tỷ giây) để thực thi một chỉ lệnh. Thật khó để cảm nhận được tốc độ đó nhanh như thế nào. Để dễ hình dung, ánh sáng chỉ đi được

khoảng một foot (0,3 mét) trong hơn một nano giây. Một cách khác để tư duy: trong khoảng thời gian âm thanh giọng nói của bạn truyền đi được 100 feet, một máy tính hiện đại có thể thực thi hàng triệu chỉ lệnh.

Để giải trí, hãy thử thực thi đoạn mã sau:

```
maxVal = int(input('Enter a positive integer: '))
i = 0
while i < maxVal:
    i = i + 1
print(i)
```

Hãy xem bạn cần nhập một số nguyên lớn đến mức nào trước khi thấy một sự tạm dừng có thể cảm nhận được trước khi kết quả được in ra.

Bài tập nhỏ: Viết một chương trình yêu cầu người dùng nhập một số nguyên và in ra hai số nguyên `root` và `pwr` sao cho $0 < pwr < 6$ và $root^{pwr}$ bằng với số nguyên mà người dùng đã nhập. Nếu không tồn tại cặp số nguyên nào như vậy, chương trình nên in ra một thông báo tương ứng.

3.2 Vòng lặp For

Các vòng lặp `while` mà chúng ta đã sử dụng từ trước đến nay mang tính khuôn mẫu rất cao. Mỗi vòng lặp đều lặp qua một dãy các số nguyên. Python cung cấp một cơ chế ngôn ngữ là vòng lặp `for`, có thể được sử dụng để đơn giản hóa các chương trình chứa loại hình lặp này.

Dạng tổng quát của câu lệnh `for` là (lưu ý các từ in nghiêng là mô tả, không phải mã nguồn thực tế):

```
for biến in dãy_giá_trị:
    khối_mã
```

Biến đứng sau `for` được liên kết với giá trị đầu tiên trong dãy, và khối mã được thực thi. Sau đó, biến được gán giá trị thứ hai trong dãy và khối mã lại được thực thi một lần nữa. Quá trình này tiếp tục cho đến khi dãy giá trị bị cạn kiệt hoặc một câu lệnh `break` được thực thi bên trong khối mã.

Dãy các giá trị liên kết với biến thường được tạo ra nhất bằng cách sử dụng hàm tích hợp `range`, hàm này trả về một loạt các số nguyên. Hàm `range` nhận ba đối số nguyên: `start` (bắt đầu), `stop` (dừng), và `step` (bước nhảy). Nó tạo ra một cấp số cộng: $start, start + step, start + 2 * step$, v.v.

- Nếu `step` dương, phần tử cuối cùng là số nguyên lớn nhất có dạng $start + i * step$ nhỏ hơn `stop`.
- Nếu `step` âm, phần tử cuối cùng là số nguyên nhỏ nhất có dạng $start + i * step$ lớn hơn `stop`.

Ví dụ: biểu thức `range(5, 40, 10)` tạo ra dãy $5, 15, 25, 35$; và `range(40, 5, -10)` tạo ra dãy $40, 30, 20, 10$. Nếu đối số đầu tiên bị bỏ qua, nó mặc định là 0; và nếu đối số cuối cùng (kích thước bước nhảy) bị bỏ qua, nó mặc định là 1. Ví dụ: `range(0, 3)` và `range(3)` đều tạo ra dãy $0, 1, 2$. Các con số trong cấp số này được tạo ra theo cơ chế "khi cần thiết" (as needed), vì vậy ngay cả các biểu thức như `range(1000000)` cũng tiêu tốn rất ít bộ nhớ.[17] Chúng ta sẽ thảo luận sâu hơn về `range` trong Mục 5.2.

Ít phổ biến hơn, chúng ta có thể chỉ định dãy giá trị để lặp trong vòng lặp `for` bằng cách sử dụng một giá trị trực tiếp (literal), ví dụ: `[0, 3, 2]`.

Xét đoạn mã:

```
x = 4
for i in range(0, x):
    print(i)
```

Kết quả in ra là: $0, 1, 2, 3$.

Bây giờ, hãy suy nghĩ về đoạn mã sau:

```
x = 4
for i in range(0, x):
    print(i)
    x = 5
```

Điều này đặt ra câu hỏi liệu việc thay đổi giá trị của x bên trong vòng lặp có ảnh hưởng đến số lần lặp hay không. Câu trả lời là không. Các đối số của hàm `range` trong dòng lệnh `for` được tính toán ngay trước lần lặp đầu tiên của vòng lặp, và không được tính toán lại cho các lần lặp tiếp theo.

Để thấy rõ cơ chế này, hãy xem xét:

```
x = 4
for j in range(x):
    for i in range(x):
        print(i)
        x = 2
```

Kết quả sẽ in ra:

```
0, 1, 2, 3
0, 1
0, 1
0, 1
```

(lần lặp đầu tiên của vòng lặp ngoài với $x = 4$)

Lý do là vì hàm `range` trong vòng lặp ngoài chỉ được tính toán một lần duy nhất, nhưng hàm `range` trong vòng lặp trong được tính toán lại mỗi khi câu lệnh `for` bên trong được chạm tới.

Mã nguồn trong Hình 3.2 thực thi lại thuật toán duyệt cạn kiệt để tìm căn bậc ba. Câu lệnh `break` trong vòng lặp `for` khiến vòng lặp kết thúc trước khi nó chạy qua hết tất cả các phần tử trong dãy giá trị đang lặp.

Hình 3.2: Sử dụng câu lệnh `for` và `break`

```
# Tìm căn bậc ba của một số lập phương hoàn chỉnh
x = int(input('Enter an integer: '))
for ans in range(0, abs(x)+1):
    if ans**3 >= abs(x):
        break
if ans**3 != abs(x):
    print(x, 'is not a perfect cube')
else:
    if x < 0:
        ans = -ans
    print('Cube root of', x, 'is', ans)
```

Câu lệnh `for` có thể được dùng kết hợp với toán tử `in` để lặp qua các ký tự của một chuỗi một cách thuận tiện. Ví dụ:

```
total = 0
for c in '12345678':
    total = total + int(c)
print(total)
```

Đoạn mã này cộng các chữ số trong chuỗi `'12345678'` và in ra tổng.

Bài tập nhỏ: Cho s là một chuỗi chứa một dãy các số thập phân ngăn cách nhau bởi dấu phẩy, ví dụ: `s = '1.23,2.4,3.123'`. Hãy viết một chương trình in ra tổng của các số trong chuỗi s.

3.3 Nghiệm xấp xỉ và Tìm kiếm nhị phân (Bisection Search)

Hãy tưởng tượng ai đó yêu cầu bạn viết một chương trình tìm căn bậc hai của một số không âm bất kỳ. Bạn nên làm gì?

Trước tiên, có lẽ bạn nên bắt đầu bằng việc yêu cầu một phát biểu bài toán (problem statement) tốt hơn. Ví dụ, chương trình nên làm gì nếu được yêu cầu tìm căn bậc hai của 2? Căn bậc hai của 2 không phải là một số hữu tỉ. Điều này có nghĩa là không có cách nào để biểu diễn chính xác giá trị của nó dưới dạng một chuỗi chữ số hữu hạn (hoặc dưới dạng số thực dấu phẩy động), vì vậy bài toán như phát biểu ban đầu là không thể giải quyết được.

Yêu cầu đúng đắn hơn là một chương trình tìm một giá trị xấp xỉ (approximation) của căn bậc hai — tức là một đáp án đủ gần với căn bậc hai thực tế để có thể sử dụng được. Chúng ta sẽ quay lại vấn đề này một cách chi tiết hơn ở phần sau của cuốn sách. Nhưng hiện tại, hãy coi "đủ gần" là một đáp án nằm trong một khoảng hằng số, gọi là epsilon (ϵ), so với đáp án thực tế.

Mã nguồn trong Hình 3.3 thực thi một thuật toán tìm xấp xỉ căn bậc hai. Nó sử dụng một toán tử mới là `+=`. Câu lệnh gán `ans += step` tương đương về mặt ngữ nghĩa với cách viết dài dòng hơn là `ans = ans + step`. Các toán tử `-=` và `*=` cũng hoạt động tương tự.

Hình 3.3: Xấp xỉ căn bậc hai bằng phương pháp duyệt cạn kiệt

```
x = 25
epsilon = 0.01
step = epsilon**2
numGuesses = 0
ans = 0.0
while abs(ans**2 - x) >= epsilon and ans <= x:
    ans += step
    numGuesses += 1
print('numGuesses =', numGuesses)
if abs(ans**2 - x) >= epsilon:
    print('Failed on square root of', x)
else:
    print(ans, 'is close to square root of', x)
```

Một lần nữa, chúng ta lại sử dụng phương pháp duyệt cạn kiệt. Lưu ý rằng phương pháp tìm căn bậc hai này không có điểm chung nào với cách tính căn bậc hai bằng bút và giấy mà bạn có thể đã học ở trường phổ thông. Thực tế thường là: cách tốt nhất để giải quyết một vấn đề bằng máy tính hoàn toàn khác với cách tiếp cận vấn đề đó bằng tay.

Khi chạy mã nguồn này, kết quả in ra là:

```
numGuesses = 49990
4.999000000001688 is close to square root of 25
```

Chúng ta có nên thất vọng vì chương trình không nhận ra 25 là một số chính phương và in ra kết quả là 5 không? Không. Chương trình đã làm đúng những gì nó được định ra để làm. Mặc dù in ra số 5 là điều tốt, nhưng nó cũng không tốt hơn việc in ra bất kỳ giá trị nào đủ gần với số 5.

Bạn nghĩ điều gì sẽ xảy ra nếu chúng ta đặt `x = 0.25`? Liệu nó có tìm được căn bậc hai gần bằng 0.5 không? Không. Nó sẽ báo cáo:

```
numGuesses = 2501
Failed on square root of 0.25
```

Duyệt cạn kiệt là một kỹ thuật tìm kiếm chỉ hoạt động nếu tập hợp các giá trị đang được tìm kiếm có bao hàm đáp án. Trong trường hợp này, chúng ta đang liệt kê các giá trị trong khoảng từ 0 đến giá trị của x. Khi x nằm trong khoảng từ 0 đến 1, căn bậc hai của x không nằm trong khoảng này. Một cách để khắc phục là thay đổi toán hạng thứ hai của `and` trong dòng đầu tiên của vòng lặp `while` thành:

```
while abs(ans**2 - x) >= epsilon and ans*ans <= x:
```

Bây giờ, hãy nghĩ về việc chương trình sẽ mất bao lâu để chạy. Số lần lặp phụ thuộc vào việc đáp án gần số 0 đến mức nào và kích thước của các bước nhảy (`step`). Nói một cách đại khái, chương trình sẽ thực thi vòng lặp `while` tối đa $x/step$ lần.

Hãy thử chạy mã nguồn với một số lớn hơn, ví dụ `x = 123456`. Nó sẽ chạy một lúc và sau đó in ra:

```
numGuesses = 3513631
Failed on square root of 123456
```

Bạn nghĩ chuyện gì đã xảy ra? Chắc chắn tồn tại một số dấu phẩy động xấp xỉ căn bậc hai của 123456 với sai số trong khoảng 0.01. Tại sao chương trình của chúng ta không tìm thấy nó? Vấn đề là kích thước bước nhảy của chúng ta quá lớn, và chương trình đã nhảy qua tất cả các đáp án phù hợp. Thử đặt `step` bằng `epsilon**3` và chạy lại chương trình. Cuối cùng nó sẽ tìm thấy câu trả lời phù hợp, nhưng có lẽ bạn sẽ không đủ kiên nhẫn để đợi nó làm xong.

Chương trình sẽ phải thực hiện khoảng bao nhiêu lần đoán? Kích thước bước nhảy sẽ là 0.000001 và căn bậc hai của 123456 là khoảng 351.36. Điều này có nghĩa là chương trình sẽ phải thực hiện trong khoảng 351.000.000 lần đoán để tìm ra câu trả lời thỏa mãn. Chúng ta có thể cố gắng tăng tốc bằng cách bắt đầu gần đáp án hơn, nhưng điều đó giả định rằng chúng ta đã biết đáp án rồi.

Đã đến lúc tìm kiếm một cách khác để tấn công bài toán này. Chúng ta cần chọn một thuật toán tốt hơn thay vì chỉ tinh chỉnh thuật toán hiện tại. Nhưng trước khi làm vậy, hãy nhìn vào một bài toán mà thoạt nhìn có vẻ hoàn toàn khác với việc tìm căn bậc hai.

Xét bài toán tìm xem một từ bắt đầu bằng một chuỗi các chữ cái cho trước có xuất hiện trong một cuốn từ điển giấy của tiếng Anh hay không. Về nguyên tắc, duyệt cạn kiệt có thể hoạt động. Bạn có thể bắt đầu từ từ đầu tiên và kiểm tra từng từ cho đến khi tìm thấy từ bắt đầu bằng chuỗi ký tự đó hoặc hết từ để kiểm tra. Nếu từ điển chứa n từ, trung bình sẽ mất $n/2$ lần kiểm tra để tìm thấy từ đó. Nếu từ đó không có trong từ điển, sẽ mất n lần kiểm tra. Tất nhiên, những ai đã từng có trải nghiệm tra cứu một từ trong một cuốn từ điển vật lý (thay vì trực tuyến) sẽ không bao giờ làm theo cách này.

May mắn thay, những người xuất bản từ điển giấy đã bỏ công sắp xếp các từ theo thứ tự từ điển (lexicographical order). Điều này cho phép chúng ta mở cuốn sách ở một trang mà chúng ta nghĩ từ đó có thể nằm ở đó (ví dụ: gần giữa cho các từ bắt đầu bằng chữ "m"). Nếu chuỗi ký tự đứng trước từ đầu tiên trên trang đó theo thứ tự từ điển, chúng ta biết phải lùi lại. Nếu chuỗi ký tự đứng sau từ cuối cùng trên trang, chúng ta biết phải tiến lên. Ngược lại, chúng ta kiểm tra xem chuỗi ký tự đó có khớp với từ nào trên trang không.

Bây giờ hãy lấy cùng ý tưởng đó và áp dụng cho bài toán tìm căn bậc hai của x. Giả sử chúng ta biết rằng một xấp xỉ tốt cho căn bậc hai của x nằm đâu đó giữa 0 và `max`. Chúng ta có thể tận dụng thực tế là các con số được sắp thứ tự toàn phần (totally ordered). Nghĩa là, đối với bất kỳ cặp số phân biệt nào n_1 và n_2, thì hoặc $n_1 < n_2$ hoặc $n_1 > n_2$. Vì vậy, chúng ta có thể coi căn bậc hai của x nằm đâu đó trên một đoạn thẳng từ 0 đến `max`:

```
0 ---------------------------------------------- max
```

và bắt đầu tìm kiếm trong khoảng đó. Vì chúng ta không nhất thiết biết bắt đầu tìm kiếm từ đâu, hãy bắt đầu ở chính giữa:

```
0 -------- guess -------- max
```

Nếu đó không phải là câu trả lời đúng (và phần lớn thời gian là như vậy), hãy hỏi xem nó quá lớn hay quá nhỏ. Nếu nó quá lớn, chúng ta biết rằng câu trả lời phải nằm ở bên trái. Nếu nó quá nhỏ, chúng ta biết câu trả lời phải nằm ở bên phải. Sau đó, chúng ta lặp lại quy trình trên khoảng nhỏ hơn đó. Hình 3.4 trình bày việc thực thi và kiểm thử thuật toán này.

Hình 3.4: Sử dụng tìm kiếm nhị phân (bisection search) để xấp xỉ căn bậc hai

```
x = 25
epsilon = 0.01
numGuesses = 0
low = 0.0
high = max(1.0, x)
ans = (high + low)/2.0
while abs(ans**2 - x) >= epsilon:
    print('low =', low, 'high =', high, 'ans =', ans)
    numGuesses += 1
    if ans**2 < x:
        low = ans
    else:
        high = ans
    ans = (high + low)/2.0
print('numGuesses =', numGuesses)
print(ans, 'is close to square root of', x)
```

Khi chạy, nó in ra:

```
low = 0.0 high = 25 ans = 12.5
low = 0.0 high = 12.5 ans = 6.25
low = 0.0 high = 6.25 ans = 3.125
low = 3.125 high = 6.25 ans = 4.6875
low = 4.6875 high = 6.25 ans = 5.46875
low = 4.6875 high = 5.46875 ans = 5.078125
low = 4.6875 high = 5.078125 ans = 4.8828125
low = 4.8828125 high = 5.078125 ans = 4.98046875
low = 4.98046875 high = 5.078125 ans = 5.029296875
low = 4.98046875 high = 5.029296875 ans = 5.0048828125
low = 4.98046875 high = 5.0048828125 ans = 4.99267578125
low = 4.99267578125 high = 5.0048828125 ans = 4.998779296875
low = 4.998779296875 high = 5.0048828125 ans = 5.0018310546875
numGuesses = 13
5.00030517578125 is close to square root of 25
```

Lưu ý rằng nó tìm ra một câu trả lời khác với thuật toán trước đó của chúng ta. Điều đó hoàn toàn ổn, vì nó vẫn đáp ứng các đặc tả của bài toán.

Quan trọng hơn, hãy lưu ý rằng tại mỗi lần lặp của vòng lặp, kích thước của không gian cần tìm kiếm giảm đi một nửa. Bởi vì nó chia đôi không gian tìm kiếm ở mỗi bước, nó được gọi là tìm kiếm nhị phân (bisection search hoặc binary search). Tìm kiếm nhị phân là một sự cải tiến khổng lồ so với thuật toán trước đó của chúng ta (vốn chỉ giảm không gian tìm kiếm một lượng rất nhỏ ở mỗi lần lặp).

Hãy thử lại với `x = 123456`. Lần này chương trình chỉ mất 30 lần đoán để tìm ra câu trả lời chấp nhận được. Còn với `x = 123456789`? Nó chỉ mất 45 lần đoán.

Không có gì đặc biệt về việc chúng ta đang sử dụng thuật toán này để tìm căn bậc hai. Ví dụ, bằng cách thay đổi một vài số 2 thành số 3, chúng ta có thể sử dụng nó để xấp xỉ căn bậc ba của một số không

âm. Trong Chương 4, chúng ta sẽ giới thiệu một cơ chế ngôn ngữ cho phép chúng ta tổng quát hóa mã nguồn này để tìm bất kỳ căn bậc n nào.

Bài tập nhỏ: Mã nguồn trong Hình 3.4 sẽ làm gì nếu câu lệnh `x = 25` được thay thế bằng `x = -25`?

Bài tập nhỏ: Cần phải thay đổi điều gì để mã nguồn trong Hình 3.4 hoạt động được cho việc tìm xấp xỉ căn bậc ba của cả số âm và số dương? (Gợi ý: hãy nghĩ về việc thay đổi `low` để đảm bảo đáp án nằm trong vùng đang được tìm kiếm.)

3.4 Một vài lưu ý khi sử dụng số thực dấu phẩy động (Floats)

Hầu hết thời gian, các con số thuộc kiểu `float` cung cấp một xấp xỉ khá tốt cho các số thực. Nhưng "hầu hết thời gian" không có nghĩa là tất cả mọi lúc, và khi chúng không làm được điều đó, nó có thể dẫn đến những hệ quả đáng ngạc nhiên. Ví dụ, hãy thử chạy đoạn mã sau:

```
x = 0.0
for i in range(10):
    x = x + 0.1
if x == 1.0:
    print(x, '= 1.0')
else:
    print(x, 'is not 1.0')
```

Có lẽ bạn, cũng giống như hầu hết mọi người, sẽ thấy ngạc nhiên khi nó in ra:

```
0.9999999999999999 is not 1.0
```

Tại sao ngay từ đầu nó lại rơi vào nhánh `else`? Để hiểu tại sao điều này xảy ra, chúng ta cần hiểu cách các số dấu phẩy động được biểu diễn trong máy tính trong suốt quá trình tính toán. Để hiểu được điều đó, chúng ta cần hiểu về số nhị phân.

Khi lần đầu tiên học về số thập phân — tức là các số cơ số 10 — bạn đã học được rằng bất kỳ số thập phân nào cũng có thể được biểu diễn bằng một chuỗi các chữ số từ 0 đến 9. Chữ số ở tận cùng bên phải là hàng 10^0, chữ số tiếp theo về bên trái là hàng 10^1, v.v. Ví dụ, chuỗi chữ số thập phân 302 biểu thị $3 \times 10^2 + 0 \times 10^1 + 2 \times 10^0$. Có bao nhiêu số khác nhau có thể được biểu diễn bằng một chuỗi có độ dài n? Một chuỗi độ dài 1 có thể biểu diễn bất kỳ số nào trong số mười số (0-9); một chuỗi độ dài 2 có thể biểu diễn một trăm số (0-99). Tổng quát hơn, với một chuỗi có độ dài n, người ta có thể biểu diễn 10^n số khác nhau.

Số nhị phân — số cơ số 2 — cũng hoạt động tương tự. Một số nhị phân được biểu diễn bằng một chuỗi các chữ số mà mỗi chữ số chỉ có thể là 0 hoặc 1. Các chữ số này thường được gọi là bit. Chữ số ở tận cùng bên phải là hàng 2^0, chữ số tiếp theo về bên trái là hàng 2^1, v.v. Ví dụ, chuỗi chữ số nhị phân 101 biểu thị $1 \times 4 + 0 \times 2 + 1 \times 1 = 5$. Có bao nhiêu số khác nhau có thể được biểu diễn bằng một chuỗi có độ dài n? Đáp án là 2^n.

Bài tập nhỏ: Giá trị thập phân tương đương của số nhị phân 10011 là bao nhiêu?

Có lẽ vì hầu hết mọi người có mười ngón tay, chúng ta dường như thích sử dụng hệ thập phân để biểu diễn các con số. Mặt khác, tất cả các hệ thống máy tính hiện đại đều biểu diễn số ở dạng nhị phân. Điều

này không phải vì máy tính sinh ra với hai ngón tay, mà là vì việc xây dựng các công tắc phần cứng (các thiết bị chỉ có thể ở một trong hai trạng thái: bật hoặc tắt) là rất dễ dàng. Việc máy tính sử dụng biểu diễn nhị phân trong khi con người sử dụng biểu diễn thập phân đôi khi có thể dẫn đến sự "mâu thuẫn trong nhận thức" (cognitive dissonance).

Trong hầu hết các ngôn ngữ lập trình hiện đại, các số không phải số nguyên được hiện thực hóa bằng một dạng biểu diễn gọi là dấu phẩy động (floating point). Hiện tại, hãy giả vờ rằng biểu diễn nội bộ là hệ thập phân. Chúng ta sẽ biểu diễn một con số dưới dạng một cặp số nguyên — các chữ số có nghĩa (significant digits) của con số đó và một số mũ (exponent). Ví dụ, số 1.949 sẽ được biểu diễn dưới dạng cặp $(1949, -3)$, tương ứng với tích 1949×10^{-3}.

Số lượng chữ số có nghĩa xác định độ chính xác (precision) mà các con số có thể được biểu diễn. Ví dụ, nếu chỉ có hai chữ số có nghĩa, số 1.949 không thể được biểu diễn chính xác. Nó sẽ phải được chuyển đổi thành một xấp xỉ nào đó của 1.949, trong trường hợp này là 1.9. Xấp xỉ đó được gọi là giá trị làm tròn (rounded value).

Máy tính hiện đại sử dụng biểu diễn nhị phân, không phải thập phân. Chúng ta biểu diễn các chữ số có nghĩa và số mũ ở dạng nhị phân thay vì thập phân và nâng cơ số 2 thay vì cơ số 10 lên lũy thừa của số mũ. Ví dụ, số 0.625 (tức 5/8) sẽ được biểu diễn dưới dạng cặp $(101, -11)$; vì 5/8 là 0.101 trong hệ nhị phân và -11 là biểu diễn nhị phân của -3, cặp $(101, -11)$ có nghĩa là $5 \times 2^{-3} = 5/8 = 0.625$.

Vậy còn phân số thập phân $1/10$, thứ mà chúng ta viết trong Python là `0.1` thì sao? Kết quả tốt nhất chúng ta có thể đạt được với bốn chữ số nhị phân có nghĩa là $(0011, -101)$. Điều này tương đương với $3/32$, tức là 0.09375. Nếu chúng ta có năm chữ số nhị phân có nghĩa, chúng ta sẽ biểu diễn 0.1 là $(11001, -1000)$, tương đương với $25/256$, tức là 0.09765625. Chúng ta cần bao nhiêu chữ số có nghĩa để có được một biểu diễn dấu phẩy động chính xác cho 0.1? Một số lượng chữ số vô hạn! Không tồn tại các số nguyên sig và exp sao cho $sig \times 2^{-exp}$ bằng đúng 0.1.

Vì vậy, cho dù Python (hay bất kỳ ngôn ngữ nào khác) chọn sử dụng bao nhiêu bit để biểu diễn các số dấu phẩy động, nó cũng sẽ chỉ có thể biểu diễn một xấp xỉ của 0.1. Trong hầu hết các bản thực thi Python, có 53 bit độ chính xác dành cho số dấu phẩy động, vì vậy các chữ số có nghĩa được lưu trữ cho số thập phân 0.1 sẽ là:

```
1100110011001100110011001100110011001100110011001100110011001
```

Giá trị này tương đương với số thập phân:

```
0.1000000000000000055511151231257827021181583404541015625
```

Rất gần với $1/10$, nhưng không chính xác là $1/10$.

Quay lại điều bí ẩn ban đầu, tại sao đoạn mã cộng 0.1 mười lần lại in ra `0.9999999999999999 is not 1.0`? Bây giờ chúng ta đã hiểu rằng phép thử `x == 1.0` cho kết quả `False` vì giá trị mà `x` liên kết không chính xác là 1.0. Chuyện gì sẽ xảy ra nếu chúng ta thêm vào cuối nhánh `else` đoạn mã `print(x == 10.0 * 0.1)`? Nó sẽ in ra `False` vì trong ít nhất một lần lặp của vòng lặp, Python đã dùng hết các chữ số có nghĩa và thực hiện việc làm tròn. Nó không giống như những gì giáo viên tiểu học đã dạy chúng ta, nhưng việc cộng 0.1 mười lần không tạo ra cùng một giá trị như việc nhân 0.1 với 10.[18]

Nhân tiện, nếu bạn muốn làm tròn một số dấu phẩy động một cách rõ ràng, hãy sử dụng hàm `round()`. Biểu thức `round(x, numDigits)` trả về số dấu phẩy động tương đương với việc làm tròn giá trị của `x`

đến `numDigits` chữ số thập phân sau dấu phẩy. Ví dụ, `print(round(2**0.5, 3))` sẽ in ra 1.414 như một xấp xỉ của căn bậc hai của 2.

Sự khác biệt giữa số thực và số dấu phẩy động có thực sự quan trọng không? May mắn thay, hầu hết thời gian thì không. Có rất ít tình huống mà 1.0 là một câu trả lời chấp nhận được nhưng 0.9999999999999999 thì không. Tuy nhiên, một điều gần như luôn đáng lo ngại là các phép thử so sánh bằng (==). Như chúng ta đã thấy, sử dụng == để so sánh hai giá trị dấu phẩy động có thể tạo ra kết quả đáng ngạc nhiên. Hầu như luôn phù hợp hơn nếu hỏi liệu hai giá trị dấu phẩy động có đủ gần nhau hay không, thay vì liệu chúng có đồng nhất hay không. Vì vậy, tốt hơn hết là nên viết `abs(x - y) < 0.0001` thay vì `x == y`.

Một điều khác cần lo lắng là sự tích tụ của sai số làm tròn (accumulation of rounding errors). Hầu hết thời gian mọi thứ diễn ra ổn thỏa vì đôi khi con số được lưu trữ trong máy tính lớn hơn một chút so với dự định, và đôi khi nó nhỏ hơn một chút. Tuy nhiên, trong một số chương trình, các sai số này có thể cùng đi theo một hướng và tích tụ dần theo thời gian.

3.5 Phương pháp Newton-Raphson

Thuật toán xấp xỉ được sử dụng phổ biến nhất thường được quy cho Isaac Newton. Nó thường được gọi là phương pháp Newton, nhưng đôi khi cũng được gọi là phương pháp Newton-Raphson.[19] Nó có thể được sử dụng để tìm nghiệm thực của nhiều loại hàm số, nhưng chúng ta sẽ chỉ xem xét nó trong ngữ cảnh tìm nghiệm thực của một đa thức một biến. Việc tổng quát hóa cho các đa thức nhiều biến là khá đơn giản cả về mặt toán học lẫn thuật toán.

Một đa thức một biến (theo quy ước, chúng ta sẽ viết biến là x) là số 0 hoặc là tổng của một số hữu hạn các hạng tử khác không, ví dụ: $3x^2 + 2x + 3$. Mỗi hạng tử (ví dụ: $3x^2$) bao gồm một hằng số (gọi là hệ số của hạng tử, trong trường hợp này là 3) nhân với biến số (x) được nâng lên một số mũ nguyên không âm (trong trường hợp này là 2). Số mũ của biến trong một hạng tử được gọi là bậc của hạng tử đó. Bậc của đa thức là bậc lớn nhất của bất kỳ hạng tử đơn lẻ nào trong đó. Một số ví dụ bao gồm: 3 (bậc 0), $2.5x + 12$ (bậc 1), và $3x^2$ (bậc 2). Ngược lại, $2/x$ và $x^{0.5}$ không phải là đa thức.

Nếu p là một đa thức và r là một số thực, chúng ta viết $p(r)$ để biểu thị giá trị của đa thức khi $x = r$. Một nghiệm của đa thức p là lời giải cho phương trình $p = 0$, tức là một số r sao cho $p(r) = 0$. Vì vậy, ví dụ, bài toán tìm xấp xỉ căn bậc hai của 24 có thể được lập công thức là tìm một số x sao cho $x^2 - 24 \approx 0$.

Newton đã chứng minh một định lý ngụ ý rằng: nếu một giá trị (gọi là *guess*) là một xấp xỉ cho nghiệm của một đa thức, thì:

$$guess - \frac{p(guess)}{p'(guess)}$$

trong đó p' là đạo hàm bậc nhất của p, sẽ là một xấp xỉ tốt hơn.[20]

Với bất kỳ hằng số k và hệ số c nào, đạo hàm bậc nhất của đa thức $cx^2 + k$ là $2cx$. Ví dụ, đạo hàm bậc nhất của $x^2 - k$ là $2x$. Do đó, chúng ta biết rằng có thể cải thiện giá trị dự đoán hiện tại (gọi là y) bằng cách chọn giá trị dự đoán tiếp theo là:

$$y - \frac{y^2 - k}{2y}$$

Đây được gọi là phương pháp xấp xỉ liên tiếp (successive approximation). Hình 3.5 trình bày đoạn mã minh họa cách sử dụng ý tưởng này để nhanh chóng tìm ra xấp xỉ của căn bậc hai.

Hình 3.5: Cài đặt phương pháp Newton-Raphson

```
# Phương pháp Newton-Raphson tìm căn bậc hai
# Tìm x sao cho x**2 - 24 nằm trong khoảng epsilon của 0
epsilon = 0.01
k = 24.0
guess = k/2.0
while abs(guess*guess - k) >= epsilon:
    guess = guess - (((guess**2) - k)/(2*guess))
print('Square root of', k, 'is about', guess)
```

Bài tập nhỏ: Hãy thêm mã nguồn vào phần cài đặt Newton-Raphson để theo dõi số lượng vòng lặp cần thiết để tìm ra nghiệm. Sử dụng đoạn mã đó như một phần của chương trình so sánh hiệu quả giữa Newton-Raphson và tìm kiếm nhị phân (bisection search). (Bạn sẽ khám phá ra rằng Newton-Raphson hiệu quả hơn nhiều).